COREJAVA笔记
1. 锁
锁分类
公平锁/非公平锁
公平锁是指多个线程按照申请锁的顺序来获取锁
非公平锁是指多个线程获取锁的顺序并不是按照申请锁的顺序，有可能后申请的线程比先申请的线程优先获取锁
可重入锁
独享锁/共享锁
独享锁是指该锁一次只能被一个线程所持有。
对于Java `ReentrantLock`而言，其是独享锁
对于`Synchronized`而言，当然是独享锁
共享锁是指该锁可被多个线程所持有对于Lock的另一个实现类`ReadWriteLock`，其读锁是共享锁，其写锁是独享锁
互斥锁/读写锁
描述独享锁/共享锁就是一种广义的说法，互斥锁/读写锁就是具体的实现
分类
互斥锁在Java中的具体实现就是`ReentrantLock`
读写锁在Java中的具体实现就是`ReadWriteLock`
乐观锁/悲观锁
描述乐观锁与悲观锁不是指具体的什么类型的锁，而是指看待并发同步的角度
适用场景
悲观锁适合写操作非常多的场景
乐观锁适合读操作非常多的场景
分段锁
分段锁其实是一种锁的设计，并不是具体的一种锁，对于`ConcurrentHashMap`而言，其并发的实现就是通过分段锁的形式来实现高效的并发操作
`ConcurrentHashMap`中的分段锁称为Segment，它即类似于HashMap（JDK7与JDK8中HashMap的实现）的结构，即内部拥有一
个Entry数组，数组中的每个元素又是一个链表；同时又是一个ReentrantLock（Segment继承了ReentrantLock)。
偏向锁/轻量级锁（自旋锁）/重量级锁
偏向锁
在大多数情况下，锁总是由同一线程多次获得，不存在多线程竞争，所以出现了偏向锁，其目标就是在
只有一个线程执行同步代码块时，降低获取锁带来的消耗，提高性能
线程执行同步代码或方法前，线程只需要判断对象头的`Mark Word`中线程`ID`与当前线程`ID`是否一致，如果一致直接执行同步代码或方法
轻量级锁（自旋锁）
自旋锁是指尝试获取锁的线程不会立即阻塞，而是采用循环的方式去尝试获取
锁，这样的好处是减少线程上下文切换的消耗，缺点是循环会消耗CPU
轻量级锁是指当锁是偏向锁的时候，被另一个线程所访问，偏向锁就会升级为轻
量级锁，其他线程会通过自旋的形式尝试获取锁，不会阻塞，提高性能
重量级锁重量级锁是依赖操作系统的`MutexLock`（互斥锁）来实现的，需要从用户态转到内核态，这
个成本非常高，这就是为什么`Java1.6`之前`Synchronized`效率低的原因
`Java1.5`以后为了减少获取锁和释放锁带来的性能消耗，引入了偏向锁和轻量级锁，
`Synchronized`的升级顺序是 「无锁-->偏向锁-->轻量级锁-->重量级锁，只会升级不会降级」
锁相关的类
synchronized
修饰方法或者代码块
对于同步方法，JVM 采用 ACC_SYNCHRONIZED 标记符来实现同步
对于同步代码块。JVM 采用 monitorenter、monitorexit 两个指令来实现同步
优化
在JDK1.6之后对对synchronized锁进行了升级，
在1.6之前synchronized直接加重量级锁，但是
1.6之后对其性能进行了优化，采用锁升级的机
制：偏向锁->自旋锁(轻量级锁)->重量级锁。
初次执行到synchronized代码块的时候，锁对象变成偏向锁。线程执行并不会主动释放锁。第二次到达代码块的时候，线程会判断之
前的线程是不是自己，如果是自己的话，由于前面没有释放锁的操作，这里也就不用重新加锁，就没有加锁、解锁的开销了。
当第二个线程过来，两个线程产生了竞争，就会从偏向锁升级成自旋锁。自旋锁竞争状态中，没有获得锁的线程，就会不断自旋判断
自己是否拿到了锁，没有拿到锁就会忙等。
三种使用方式
- 修饰普通函数，监视器锁（`monitor`）便是对象实例（`this`）
- 修饰静态静态函数，视器锁（`monitor`）便是对象的`Class`实例（每个对象只有一个`Class`实例），又因为Class的相关数据存储在永久
带PermGen（jdk1.8则是metaspace），永久带是全局共享的，因此静态方法锁相当于类的一个全局锁，会锁所有调用该方法的线程
- 修饰代码块，监视器锁（`monitor`）是指定对象实例
原理
在对synchronized修饰的方法所在的类进行反编译后，我们发现存在`monitorenter`与`monitorexit`指令（获取锁、释放锁）。
`monitorenter`指令插入到同步代码块的开始位置，`monitorexit`指令插入到同步代码块的结束位置，`JVM`需要保证每一个 `monitorenter`都有
`monitorexit`与之对应。
synchronized的对象锁，其指针指向的是一个monitor对象（由C++实现）的起始地址。每个对象实例都会有一个 monitor。其中monitor可以与对象一
起创建、销毁；亦或者当线程试图获取对象锁时自动生成。任何对象都有一个监视器锁（`monitor`）关联，线程执行`monitorenter`指令时尝试获取
`monitor`的所有权
ObjectMonitor() {
 _count = 0; //记录个数
 _owner = NULL; // 运行的线程
 //两个队列
 _WaitSet = NULL; //调用 wait 方法会被加入到_WaitSet
 _EntryList = NULL ; //锁竞争失败，会被加入到该列表。线程处于阻塞状态（Blocked）
}
`_owner`指向持有ObjectMonitor对象的线程。当多个线程同时访问一段同步代码时，会先存放到 _EntryList 集合中，接下来当线
程获取到对象的monitor时，就会把_owner变量设置为当前线程。同时count变量+1。如果线程调用wait() 方法，就会释放当前持有
的monitor，那么_owner变量就会被置为null，同时_count减1，并且该线程进入 WaitSet集合中，等待下一次被唤醒
两个线程是否可以同是访问一个对象的两个不同
的synchronized方法?
多个线程访问同一个类的synchronized方法时, 都是串行执行的 ! 就算有多个cpu也不例外 ! synchronized方法使用了类java的内置锁,
即锁住的是方法所属对象本身. 同一个锁某个时刻只能被一个执行线程所获取, 因此其他线程都得等待锁的释放. 因此就算你有多余的
cpu可以执行, 但是你没有锁, 所以你还是不能进入synchronized方法执行
一个类的static构造方法加上synchronized之后
的锁的影响
synchronized是一般对类的当前实例（当前对象）进行加锁，防止其他线程同时访问该类的该实例的所有
synchronized块，注意这里是“类的当前实例”，类的两个不同实例就没有这种约束了
synchronized相当于 this.synchronized，而static synchronized相当于AClass.synchronized
AQS
描述 AbstractQueuedSynchronizer简称AQS，它是一个用于构建锁和同步容器的框架。AQS定义了一套多线程访问共享资源的同步器框架，许多同步
类实现都依赖于它，如常用的ReentrantLock/Semaphore/CountDownLatch
核心思想
如果被请求的共享资源空闲，则将当前请求资源的线程设置为有效的工作线程，并将共享资源设置为锁
定状态，如果被请求的共享资源被占用，那么就需要一套线程阻塞等待以及被唤醒时锁分配的机制，这
个机制AQS是用CLH队列锁实现的，即将暂时获取不到锁的线程加入到队列中。
CLH(Craig,Landin,and Hagersten)队列是一个虚拟的双向队列（虚拟的双向队列即不存在队
列实例，仅存在结点之间的关联关系）。AQS是将每条请求共享资源的线程封装成一个CLH锁
队列的一个结点（Node）来实现锁的分配
AQS的三个关键
state state用来标记共享变量的状态，一般用volatile来修饰。AQS中用state属性表示锁，如果能成功将state属性通过CAS操作从0设置成1即获取了锁
queue
当线程请求锁失败后，将
线程包装为一个Node，加
入到queue中，等待后续
的唤醒操作。
AQS使用CLH队列锁实现
是什么
CLH(Craig,Landin,and Hagersten)队列是一个
虚拟的双向队列(虚拟的双向队列即不存在队列实
例，仅存在结点之间的关联关系)
CLH 锁的队列是一个虚拟的 FIFO 队列（先进先出）。每个节点包
含一个标志位，通常是一个布尔值，表示该线程是否在等待锁。队
列的尾部保持一个指向最后一个节点的引用
之所以叫隐形的链表是因为这些结点之间没有明显的 next 指针，而
是通过 myPred 所指向的结点的变化情况来影响 myNode 的行为
一种是基于逻辑队列非线程饥饿的一种自旋公平锁
原理
队列头节点称作“哨兵节点”或者
“哑节点”，它不与任何线程关联。
其他的节点与等待线程关联，每
个节点维护一个等待状态
waitStatus。
锁请求
当一个线程想要获得锁时，它创建一个表示自己的节点，并将节点的状态设置为锁定（例如，设置一个布尔标志为真）
这个新节点被插入到队列的尾部
线程检查前一个节点的状态。如果前一个节点的状态表示锁定，则当前线程进入自旋
（busy-waiting），不断检查前一个节点的状态
锁释放
当一个线程释放锁时，它将自己的节点状态设置为非锁定（例如，布尔标志设置为假）
然后它将队列的前指针移动到自己的节点，从而允许后面的节点成为新的队列头部，即队列的新锁持有者
特点
非阻塞结构 CLH 锁是一种非阻塞的锁结构，这意味着线程在等待获取锁时不会被操作系统挂
起，而是持续检查锁的状态，这称为自旋
公平性由于其队列结构，CLH 锁天然地保证了公平性，因为获取锁的顺序就是请求锁的顺序
空间效率 CLH 锁相对节省空间，因为每个节点仅需存储一个状态标志和一个指向前一个节点的链接
适用性它适用于那些锁持有时间较短的场景，因为线程持续自旋可能会消耗大量的CPU资源
CAS 利用CAS来修改state和queue中的入队操作等
AQS内部实现了两个队列
等待队列（Wait Queue）
当线程获取资源失败之后，就进入同步队列的尾部保持自旋等
待，不断判断自己是否是链表的头节点，如果是头节点，就不
断尝试获取资源，获取成功后则退出同步队列
- 条件队列是为Lock实现的一个基础同步器，并且一个线程可能会有
多个条件队列，只有在使用了Condition才会存在条件队列
- 同步队列和条件队列都是由一个个Node组成的。AQS内部有一个静态内部类Node
作用实现独占模式（如ReentrantLock）和共享模式（如Semaphore、CountDownLatch）的同步器
条件队列（Condition Queue）
条件队列与Java中的Condition对象相关联，是基于等待队列实现的。当线程需要等待某个特定的条件成立时，它会被加入到条件队列中。条件队
列不直接参与同步状态的获取与释放，而是用于线程间的协调和通信
条件等待与通知线程调用Condition的await()方法后，会被加入到条件队列中等待。当其他线程调用signal()或signalAll()方法时，等待在条件队列中
的线程会被转移回等待队列，然后尝试重新获取与该Condition相关联的锁
与等待队列的关联条件队列与等待队列协同工作，确保线程在正确的时机被唤醒。条件队列中的线程必须先移回等待队列，才能参与同步状态的竞争
AQS定义两种资源共享方式
Exclusive（独占） ReentrantLock
- 公平锁：按照线程在队列中的排队顺序，先到者先拿到锁
- 非公平锁：当线程要获取锁时，无视队列顺序直接去抢锁，谁抢到就是谁的
Share（共享）多个线程可同时执行，如Semaphore、CountDownLatch、 CyclicBarrier、ReadWriteLock
设计模式模板方法自定义同步器实现时主要实现以下几种方法
isHeldExclusively()：该线程是否正在独占资源
tryAcquire(int)：独占方式。尝试获取资源，成功则返回true，失败则返回false
tryRelease(int)：独占方式。尝试释放资源，成功则返回true，失败则返回false
tryAcquireShared(int)：共享方式。尝试获取资源。负数表示失败；0表示成功，但没有剩余可用资源；正数表示成功，且有剩余资源
tryReleaseShared(int)：共享方式。尝试释放资源，如果释放后允许唤醒后续等待结点返回true，否则返回false
AQS的延伸应用
ReentrantLock state初始化为0，表示未锁定状态
CountDownLatch 任务分为N个子线程去执行，state也初始化为N
（注意N要与线程个数一致）
这N个子线程是并行执行的，每个子线程执行完后countDown()一次，
state会CAS减1。等到所有子线程都执行完后(即state=0)，会unpark()
主调用线程，然后主调用线程就会从await()函数返回，继续后余动作
Semaphore
state初始值假设为N，后续每tryAcquire()一
次，state会CAS减1
A、B、C、D线程同时争抢资源，目前卡槽大小为2，若A、B正在执行且未执行完，那
么C、D线程在门外等着，一旦A、B有1个执行完了，那么C、D就会竞争看谁先执行
ReentrantLock
基本实现先通过CAS尝试获取锁。如果此时已经有线程占据了锁，那就加入AQS队列并且被挂起。
当锁被释放之后，排在CLH队列队首的线程会被唤醒，然后CAS再次尝试获取锁
公平锁和非公平锁不同公平锁在获取锁的时候，不会先去检查state状态，而是直接执行aqcuire(1)
reentrantLock#lockInterruptibly
lockInterruptibly()方法比较特殊，当通过这个方法去获
取锁时，如果其他线程正在等待获取锁，则这个线程能够
响应中断，即中断线程的等待状态
当两个线程同时通过lock.lockInterruptibly()想获取某个锁时，假若此时线程A获取到了
锁，而线程B只有等待，那么对线程B调用threadB.interrupt()方法能够中断线程B的等待过
程，线程B想不等的时候就可以不等
ReentrantReadWriteLock
一把叫读锁，一把叫写锁。读读不互斥。读写，写写互斥
AQS中的state变量。高16位表示读锁的数量，
低16位表示写锁的数量怎么表示每个线程持有的读锁数量
state变量的高16位表示的是所有线程持有的读锁数量总
量。如果要表示每个线程各自持有的变量，就需要用到
ThreadLocal。即源码中的这个变量
延伸 Mysql锁状态的维护
InnoDB rw_lock_t 仅使用一个64 bit整型
的lock_word就维护了绝大部分的锁状
态，其取值含义如下图所示。
Semaphore 可以用来限制能同时访问共享资源的线程上限，它内部维护了一个许可的变量，也就是线程许可的数量
CountDownLatch 允许一个或者多个线程去等待其他线程完成操作内部维护了一个计数器，该计数器初始值为 N，代表需要等待的线程数目，当一个线程完成了需要等待的任务后，就
会调用 countDown() 方法将计数器减 1，当计数器的值为 0 时，等待的线程就会开始执行。
CyclicBarrier
Condition
用来替代传统的Object的wait()、notify()实现线程间的协作，使用Condition的await()、signal()这种方式实现线程间协作更加安全和高效
Condition对象是由Lock对象创建出来的
比较
AQS与Synchronized有一个区别
Synchronized中所有阻塞的线程都集中到了一个队列中，当有线程释放锁的时候，会将所有线程都唤醒。因为
有些线程需要的条件并不满足，但是也被唤醒了，这种线程唤醒了即使得到了锁也无法执行，这就会浪费了资源
AQS排队唤醒(公平锁)
synchronized和reentrantLock区别(非超响)
响应中断 lockInterruptbly()
是什么它允许一个线程获取锁的过程中对中断做出响应。这个功能使得线程在等待获取锁的过程中，如果被中断，能
够立即响应这个中断事件，而不是像普通的锁那样可能需要长时间等待或者永远无法获取锁
工作原理
当一个线程A调用lockInterruptibly()方法尝试获
取一个锁时，如果该锁被其他线程持有，线程A
会被阻塞，等待锁释放。在这种阻塞状态下，如
果其他线程对线程A调用了interrupt()方法，那
么两种情况可能发生
如果线程A正在lockInterruptibly()方法中等待锁：线程A会捕获到这个中断异常
（InterruptedException），随后可以选择停止等待并退出锁获取的过程，从而处理中断。这种
机制对于取消长时间运行的操作非常有用，特别是当锁的等待时间未知或可能非常长时
如果线程A已经持有锁或在调用interrupt()时并未处于等待状态：中断会被设置（线程的中断状
态被设为true），但不会立即影响到线程的执行，直到线程下一次进入一个可中断的阻塞状态
非阻塞式的获取锁 tryLock() 尝试获取，获取不到不会阻塞，直接返回 lock 方法如果获取不到锁会一直阻塞等待
超时 tryLock(long time, timeUnit) 给定一个时间限制，如果一段时间内没获取到，不是进入阻塞状态，同样直接返回
wait/notify和Condition对比
锁用到的数据结构
可重入锁
特点
线程拥有性重入锁，指的是以线程为单位，当一个线程获取对象锁之后，这个线程可以再次获取本对象上的锁，而其他的线程是不可以的
重复获取锁的拥有者线程可以再次获取该锁而不会导致自己被阻塞。这是可重入性的核心特点。如果一个锁不是可
重入的，那么即使是锁的拥有者线程也会因为再次尝试获取锁而被阻塞
计数机制可重入锁内部维护着一个计数器，用于跟踪锁被同一个线程重复获取的次数
完全释放只有当计数器回到零时，锁才被完全释放
为什么可重入锁可以防止死锁？ 可重入锁本身并不能直接防止所有类型的死锁，但它可
以防止特定情况下由于重复获取同一锁导致的死锁。
在不使用可重入锁的情况下，如果一个线程已经持有某个锁，然后在持有锁的状态下再次尝试获取同一个锁，这
个线程会因为等待它自己释放锁而永久阻塞，这种情况可以视为自身死锁。
由于可重入锁允许线程重入任何已经由自己持有的锁同步的代码块，因此它避免了这种情况。当线程再次请求已
经持有的锁时，可重入锁会识别请求来自同一个线程，并允许这种重入，而不是让线程在自己持有的锁上阻塞。
synchronized 和 ReentrantLock 都是可重入锁
线程间共享变量
死锁的发生条件(互持不抢循)
互斥条件资源不能被多个线程同时占用
持有并等待条件一个线程持有至少一个资源，并等待获取额外的资源
不可抢占条件资源不能被强制从一个线程中抢
循环等待条件存在一个线程-资源的循环链，每个线程都在等待下一个线程所持有的资源
操作系统同步原语
1. 互斥锁（Mutexes）互斥锁提供了对共享资源的互斥访问。当一个线程获得互斥锁时，其他任何试图再次获取该锁的线程将被阻塞，直到锁被释放。
2. 信号量（Semaphores）
信号量是一个计数器，表示可用资源的数量。线程在进入临界区前必须获取信号量（计数器减一），退出时释放信号量（计数器加一）。用
于控制多个线程对共享资源的访问。它允许多个线程同时访问有限数量的资源实例。信号量主要用于实现资源计数和线程同步。
3. 事件（Events）事件是一种通信机制，它允许一个线程信号化其他一个或多个线程关于某个特定事件的发生。事件可以是自动重置或手动重置。
4. 条件变量（Condition Variables）
条件变量允许线程在某些预定条件未满足时挂起，直到另一个线程改变了条件并通知条件变量。条件变量常常与互斥锁一起使
用，以避免复杂的竞争条件。
5. 临界区（Critical Sections）临界区是用于保护代码区域免受多线程同时执行的影响的一种方法。它们通常依靠互斥锁来实现。
6. 读写锁（Read-Write Locks）读写锁允许多个读取者同时访问资源，但写入者的访问是互斥的。这种锁对于读多写少的资源共享模式非常有效。
7. 自旋锁（Spinlocks）
自旋锁是一种在等待锁的释放期间使线程处于忙等（busy-waiting）状态的锁。与阻塞线程相比，自旋锁避免了线程上下文切换
的开销，适用于锁持有时间极短的情况。
8. 屏障（Barriers）屏障用于多线程程序中，确保在所有线程都达到某个点后才能继续执行。屏障通常用于同步并行任务的执行阶段。
9. 票据锁（Ticket Locks）票据锁是一种公平的同步机制，每个线程在尝试获取锁时都会获得一个唯一的票据号。锁会按照票据号的顺序依次授予线程。
10. 顺序锁（Seqlocks）顺序锁是一种特别设计的锁，用于读操作远多于写操作的场景。写者会阻塞读者和其他写者，但多个读者可以并发无锁访问数据。
2. 集合
队列
没有实现的阻塞
ArrayDeque 是什么一个使用动态数组实现的双端队列，允许在两端插入和移除元素，但它不是线程安全的，也没有实现BlockingQueue接口，因此它是非阻塞的
PriorityQueue 是什么一个基于优先级堆的队列，元素根据其自然顺序或者构造时指定的Comparator被排序，同样不是线程安全的，没有实现BlockingQueue接口，因此也是非阻塞的
ConcurrentLinkedQueue 是什么一个基于链接节点的并发队列，支持在高并发环境下的无锁算法，实现了Queue接口，但没有实现BlockingQueue接口，因此是非阻塞的
CopyOnWriteArrayList 是什么是一个非阻塞的线程安全的列表。它通过一种称为写时复制（Copy-On-Write）的机制来实现线程安全
LinkedList
是什么 LinkedList内部封装的是双向链表数据结构
LinkedList#get为什么慢一半前正序遍历、一半后倒序遍历
实现阻塞
ArrayBlockingQueue
是什么一个基于数组结构的有界阻塞队列，按照FIFO（先进先出）原则对元素进行排序，是阻塞的
原理
数组实现的线程安全的有界的阻塞队列，此队列按 FIFO（先进先出）原则对元素进行排序
ArrayBlockingQueue利用 ReentrantLock 实现线程安全，是(非)公平访问是通过ReentrantLock的是(非)公平锁来实现的
实现 ArrayBlockingQueue的是(非)公平访问是通过
ReentrantLock的是(非)公平锁来实现的
public ArrayBlockingQueue(int capacity, boolean fair) {
 //...
 this.items = new Object[capacity];
 lock = new ReentrantLock(fair); //<---- ReentrantLock的是(非)公平锁来实现的
 notEmpty = lock.newCondition(); //堵塞和唤醒使用Condition，wait,notify已弃用
 notFull = lock.newCondition();
}
几个重要的方法
- add(E e)：把 e 加到 BlockingQueue 里，即如果 BlockingQueue 可以容纳，则返回 true，否则报异常
- offer(E e)：表示如果可能的话，将 e 加到 BlockingQueue 里，即如果 BlockingQueue 可以容纳，则返回 true，否则返回 false
- put(E e)：把 e 加到 BlockingQueue 里，如果 BlockQueue
没有空间，则调用此方法的线程被阻断直到 BlockingQueue 里
面有空间再继续
put在队列满的时候，通过`notFull.await();`堵
塞队列，然后在消费者从队列中消费后，通过
`notFull.signal()`唤醒堵塞
public void put(E e) throws InterruptedException {
 Objects.requireNonNull(e);
 final ReentrantLock lock = this.lock; //<------ 利用 ReentrantLock 实现线程安全
 lock.lockInterruptibly(); //<------ 获取锁
 try {
 while (count == items.length)
 notFull.await(); //<------ notFull堵塞
 enqueue(e);
 } finally {
 lock.unlock();
 }
 }
- poll(time)：取走 BlockingQueue 里排在首位的对象，若不能立即取出，则可以等 time 参数规定的时间，取不到时返回 null
- take()：取走 BlockingQueue 里排在首位的对象，若 BlockingQueue 为空，阻断进入等待状态直到 Blocking 有新的对象被加入为止 take()和poll()的区别就是当队列为空的时候，take()会阻塞
- remainingCapacity()：剩余可用的大小。等于初始容量减去当前的 size
LinkedBlockingQueue
是什么一个基于链表的双端阻塞队列，允许在两端插入和移除元素，是阻塞的
内部由单链表实现，只能从head取元素，从tail添加元素。添加元素和获取元素都有独立的锁，即LinkedBlockingQueue是读写分离的，读写操作可以并行执行
同样提基于ReentrantLock来实现锁的机制
锁分离的方式进行控制减少了take和put之间的锁竞争
private final ReentrantLock takeLock = new ReentrantLock();
private final Condition notEmpty = takeLock.newCondition();
private final ReentrantLock putLock = new ReentrantLock();
private final Condition notFull = putLock.newCondition();
1. LinkedBlockingQueue是通过锁分离的方式进行控制，减少了take和put
之间的锁竞争
2. LinkedBlockingQueue是通过链表的方式实现，所以进行锁分离时不会冲
突，因为入队和出队分别作用于队尾和队首
3. 内部采用了原子操作类（CAS: AtomicInteger count）进行控制链表长度
4. 入队后，如果之前队列为空时，会通知take方法，队列已有数据可进行
take，反之，出队后，队列之前已满，则通知put方法，队列已有空闲位置
可进行put操作
PriorityBlockingQueue 是什么一个支持优先级排序的无界阻塞队列，元素根据其自然排序或者构造时提供的Comparator进行排序，是阻塞的
DelayQueue 是什么一个无界的BlockingQueue实现，用于放置实现了Delayed接口的元素，其中的元素只能在其指定的延迟时间过后才能取出，是阻塞的
LinkedBlockingDeque 是什么一个基于链表的双端阻塞队列，允许在两端插入和移除元素，是阻塞的
SynchronousQueue 是什么并发同步阻塞队列，一个利用 BlockingQueue 接口的简单聚集（rendezvous）机制，一个不存储元素
的阻塞队列，每个插入操作必须等待一个相应的删除操作，反之亦然
ArrayBlockingQueue和
LinkedBlockingQueue区别
1. 队列大小有所不同，ArrayBlockingQueue是有界的初始化必须指定大小，而LinkedBlockingQueue可以是有界的也可以是无界的(Integer.MAX_VALUE)，对于
后者而言，当添加速度大于移除速度时，在无界的情况下，可能会造成内存溢出等问题
2. 数据存储容器不同，ArrayBlockingQueue基于数组，LinkedBlockingQueue基于链表，是以Node节点作为连接对象
3. 由于ArrayBlockingQueue采用的是数组的存储容器，因此在插入或删除元素时不会产生或销毁任何额外的对象实例，而LinkedBlockingQueue则会生成一个额
外的Node对象。这可能在长时间内需要高效并发地处理大批量数据的时，对于GC可能存在较大影响
4. 两者的实现队列添加或移除的锁不一样，ArrayBlockingQueue实现的队列中的锁是没有分离的，即添加操作和移除操作采用的同一个ReenterLock锁，而
LinkedBlockingQueue实现的队列中的锁是分离的，其添加采用的是putLock，移除采用的则是takeLock，这样能大大提高队列的吞吐量，也意味着在高并发的
情况下生产者和消费者可以并行地操作队列中的数据，以此来提高整个队列的并发性能
Map
HashMap
存储结构数组 + 链表+红黑树
扩容
什么时候扩容当hashmap中的元素个数超过数组大小*loadFactor时，就会进行数组扩容，loadFactor的默认值为0.75，也就是说，默认情况下，数组大小为16，那么
当hashmap中元素个数超过16*0.75=12的时候，就把数组的大小扩展为2*16=32，即扩大一倍
什么时候扩容 h & (length - 1); //hash值&表长度
使用的是2次幂的扩展(指长度扩为原来2倍)，所以，元素的位置要么是在原位置，要么是在原位置再移动2次幂的位置。扩充
HashMap的时候，不需要像JDK1.7的实现那样重新计算hash，只需要看看原来的hash值新增的那个bit是1还是0就好了，是0的
话索引没变，是1的话索引变成“原索引+oldCap”。oldCap是扩容的大小，由于新增的1bit是0还是1可以认为是随机的，因此
resize的过程，均匀的把之前的冲突的节点分散到新的bucket了
特点
改进红黑树
定义
节点颜色节点可以是红色或黑色节点可以是红色或黑色
根节点颜色根节点必须是黑色
叶子节点所有叶子节点（即空节点）都是黑色的
红色节点的子节点如果一个节点是红色的，那么它的两个子节点必须是黑色的
路径长度从任一节点到其叶子节点的所有路径上包含相同数目的黑色节点
性质它接近平衡，这意味着它的左右子树高度差不会太大，通常不会超过1，但也不会超过一倍。这种性质有
助于减少树在动态更新时的旋转操作，从而保持树的平衡，减少查找和插入操作的时间复杂度
红黑树的查询效率 O(logn)
jdk1.8中的应用在链表长度大于等于8的时候，转为红黑树插入；当小于等于6的时候转成链表
QA 准备用HashMap存1w条数据，构造
时传1W还会触发扩容吗？
那么当我们从外部传递进来 1w 时，实际上经过 tableSizeFor() 方法处理之后，就会变成 2 的 14 次幂 16384，
再算上负载因子 0.75f，实际在不触发扩容的前提下，可存储的数据容量是 12288（16384 * 0.75f）
如果HashMap 初始容量指定为 1000，扩容的重要依据扩容阈值会在 resize() 中调整为 768（1024 * 0.75）。它
是不足以承载 1000 条数据的，最终在存够 1k 条数据之前，还会触发一次动态扩容
hashcode
1. hashCode 计算
int h = key.hashCode(); 键的 hashCode 方法：首先，HashMap 调用键
对象的 hashCode 方法来获取原始的哈希码
h = h ^ (h >>> 16); 扰动函数：为了减少哈希碰撞的可能性，JDK 1.8 使用了一个扰动函数（perturbation function）对原始的哈希码
进行处理。扰动函数的目的是为了在高位和低位之间混淆哈希值，从而使得哈希码更加均匀地分布在哈希表中。
2. 哈希桶索引计算 int index = (n - 1) & hash;
经过扰动函数处理后的哈希码用于计算哈希桶的索引：
其中，n 是哈希表的容量（通常是2的幂），hash 是经过扰动函数处理后的哈希码。这个计算
方式利用了位运算的高效性，同时确保哈希码可以均匀地分布在哈希表的所有桶中
3. 树化和链化
if (binCount >= TREEIFY_THRESHOLD - 1) //
-1 for 1st
 treeifyBin(tab, hash);
红黑树来优化哈希表的性能。在高碰撞率的情况下，如果单个桶中
的元素数量超过一定阈值（默认是8），则链表将转换为红黑树
ConcurrentHashMap
jdk1.7
数据结构数组+Segment+分段锁
原理
ConcurrentHashMap 为了提高本身的并发能力，在内部采用了一个叫做 Segment 的结构，一个 Segment 其
实就是一个类 Hash Table 的结构，Segment 内部维护了一个链表数组
ConcurrentHashMap采用了二次hash的方式，第一次hash将key映射到对应的segment，而第二次hash则是
映射到segment的不同桶(bucket)中
为什么要用二次hash，主要原因是为了构造分离锁，使得对于map的修改不会锁住整个容器，提高并发能力
jdk1.8
数据结构数组+链表+红黑树
原理
在 JAVA8 中它摒弃 Segment（锁段）的概念，而是启用了一种全新的方式实现，利用 CAS 算法和红黑树
尽管放弃了 Segment，JDK 8 中的 ConcurrentHashMap 仍然使用桶来存储数据。桶是哈希表中的基本组成部分，每个桶可以存储一个或多个键值对。在 JDK 8 中，当桶中
的元素较少时，这些元素以链表的形式存储；当某个桶中的元素数量超过特定阈值（默认为 8）时，链表将转换为红黑树，以提高查找效率
虽然在 JDK1.8 中还能看到 Segment 的数据结构，但是已经简化了属性，只是为了兼容旧版本
锁粒度锁的是第一个头节点 Node，使用synchronized
treeMap TreeMap 继承于AbstractMap ，其是一个有序的key-value集合，内部**基于红黑树**实现
时间轮队列（Time Wheel Queue） TODO
3. 并发类
volatile
特性
可见性（Visibility）
volatile变量每次被线程访问时，都强迫线程从主内存中重读该变量的最新值，而当该变量发生修改变化时，也
会强迫线程将最新的值刷新回主内存中。这样一来，不同的线程都能及时的看到该变量的最新值。
防止指令重排
volatile变量之前的操作不会被重
排到volatile变量之后，反之亦然
这个特性使得volatile变量成为一种轻量级的同步机制，尤其在实
现双重检查锁定（double-checked locking）等模式时非常有用
指令重排
编译器优化 Java编译器在将代码编译成字节码时，可能会重新排序指令以提高性能
JIT编译时 Java虚拟机在执行代码时，JIT（Just-In-Time）编译器也可能会对指令进行重排
处理器重排现代的多核处理器为了提高执行效率，可能会在内部对输入的指令进行重新排序
一个变量被 volatile
修饰后，JVM 会为
我们做两件事
StoreLoad 屏障
在每个 volatile 写操作前插入 StoreStore 屏
障，在写操作后插入 StoreLoad 屏障。
（StoreStore-写-StoreLoad）
LoadStore屏障
在每个 volatile 读操作前插入 LoadLoad 屏障，
在读操作后插入LoadStore屏障。（LoadLoad-读-LoadStore）
例子
状态标志 private volatile boolean running = true;
单例双重检查锁定
private static volatile Singleton instance;
以确保单例对象的创建过程是安全的。volatile 变量禁止指令重排序，这对于延
迟初始化和单例模式非常重要，确保在对象初始化完成之前不会发布对象的引用
由于 JVM 的指令重排序，对象的赋值操作可能在构造函数完成所有初始
化之前发生。这意味着一个线程可能看到一个尚未完全构造的对象。
volatile 保证了这个赋值操作不会在对象构造器执行完成之前发生。这就确保
了其他线程在访问 instance 变量时，看到的是一个完全初始化的对象
最近一次写入 private volatile Map<String, String> configOptions; configOptions 使用 volatile 修饰，这确保了所有线程都能看到最近一次写入的配置
注意 volatile 是不安全的，volatile不能保证变量更改的原子性如number++，这个操作实际上是三个操作的集合（读取number，number加1，将新的值写回number）
AtomicInteger
原理
CAS 核心是利用 CAS 操作。CAS 是一种硬件对并发操作共享数据的支持。CAS 操作包含三个操作数 —— 内存位置（在这里是变
量的值）、预期原值和新值。CAS 仅在位置的当前值与预期原值相匹配时，才将位置的值更新为新值。这个操作是原子的。
无锁算法由于 CAS 是一种底层原子指令，它不需要使用传统的锁机制来保证线程安全，这减少了线程切换和调度的开销
忙等待当多个线程试图同时更新 AtomicInteger 的值时，如果 CAS 失败，线程可能会循环并重试。这是一种忙
等待策略，可以避免线程挂起，但在高竞争的情况下可能会导致CPU使用率增加
场景更适合低到中等并发的场景，或者除了累加操作外还需要进行其他类型的原子操作
ABA问题 Java 提供了 AtomicStampedReference 工具类，通过为引用建立类似版本号（stamp）的方式，来保证 CAS 的正确性
LongAdder
原理
分段累加当多个线程同时更新时，它们不会竞争同一个原子变量，而是在内部的多个单元（称为 Cell）上分散累加。每个线程累加自己的单元，最后将这些单元的值汇总
动态单元数组当线程数量较少且冲突不多时，所有累加操作可能只在一个基础计数器上进行。但在冲突增多的情况下，LongAdder 动态地创建多个单元来减少竞争
减少CAS冲突通过将操作分散到不同的单元，LongAdder 减少了对单一原子变量的CAS操作冲突，从而提高了性能，特别是在高并发情况下
场景适用于高并发环境下的累加操作，特别是当更新操作的数量远远大于读取操作的数量时。但如果你需要频繁地读取累加器的当前值，LongAdder 可
能不如 AtomicInteger 高效，因为它需要汇总多个单元的值
Condition
是什么 Condition接口是Java并发包java.util.concurrent.locks中的一个重要组件，它提供了一种比对象监视器（Object Monitor）更灵活的线程协调机制
原理工作原理类似于传统的Object监视器方法（wait(), notify(), notifyAll()），但它提供了更丰富的功能 Condition是与特定的Lock实例绑定的
使用
Condition时，一个Lock对象可以绑定多个Condition对象。这允许实现更细粒度的线程协调，比如在一个锁上实现多个等待集（wait-sets）
Lock lock = new ReentrantLock();
Condition condition = lock.newCondition(); condition.await();
使用场景 Condition在需要多个条件队列或者更精细的线
程协调控制时非常有用
在生产者-消费者问题中，可以用两个Condition实例分别控制“非空”和“非满”条件，这样生
产者和消费者就可以在不同的条件上等待和通知，从而更高效地协调它们的操作
注意事项
在调用Condition的await()方法之前，线程必须获得相关的Lock。
await()方法会释放锁，并使线程进入等待状态，直到其他线程调用signal()或signalAll()方法。
调用signal()或signalAll()方法后，线程不会立即从await()返回，需要重新获得锁。
Lock和Condition提供了比传统Object监视器方法更灵活的操作，但也更加复杂，需要谨慎使用以避免死锁或竞争条件等问题。
优势
与Lock结合 ReentrantLock允许尝试非阻塞地获取锁、尝试获取锁并可中断，以及公平性选择等
多个等待集（Wait-sets）每个Lock对象可以有多个Condition对象，这意味着可以有多个等待集。在使用synchronized时，每个对象只有一个内置的等待集
分离锁的获取与条件等待使用Condition可以在不同的作用范围内分别处理锁的获取和释放以及条件的等待和通知。这提供了更大的灵活性，可以减少锁持有时间，从而提高效率
ThreadLocal
是什么 ThreadLocal提供了线程的局部变量，在当前线程中有属于自己的变量，实现不同线程间的数据隔离，线程间的数据互不干扰
思想空间换时间当多线程访问一个共享变量时,先拷贝一份共享变量为自己的本地变量,后进行修改等操作
数据结构
结构图
每个线程都有一个
ThreadLocal.ThreadLocalMap对象（属
性名为：threadLocals），
ThreadLocalMap类是ThreadLocal类的一
个静态内部类，用于线程对象存储线程独
享的变量副本
只有当Thread被回收时，这个value才有被回收的机会，否则，只要线程不退
出，value总是会存在一个强引用，对于线程池来说，大部分线程会一直存在
在系统的整个生命周期内，那样的话，就会造成value对象出现泄漏的可能
ThreadLocalMap
ThreadLocalMap本身不是Map，但是可以实现以key-value的形式存储
线程的局部变量。与Map类似，ThreadLocalMap中将键值对的关系封装为
了一个Entry对象，Entry是ThreadLocalMap的静态内部类
ThreadLocalMap内部维护着Entry数组，每个Entry
代表一个完整的对象，key是当前 threadLocal 实
例，value是ThreadLocal的泛型值。每个Entry弱引用
实现
代码
ThreadLocalMap 定义
static class ThreadLocalMap {
 static class Entry extends WeakReference<ThreadLocal<?>> {
 Entry(ThreadLocal<?> k, Object v) {
 super(k);
 value = v;
 }
 }
 private static final int INITIAL_CAPACITY = 16;
 private Entry[] table;
 private int size = 0;
 private int threshold;
 //.....
 ThreadLocalMap(ThreadLocal<?> firstKey, Object firstValue) {
 table = new Entry[INITIAL_CAPACITY];
 int i = firstKey.threadLocalHashCode & (INITIAL_CAPACITY - 1);
 table[i] = new Entry(firstKey, firstValue);
 size = 1;
 setThreshold(INITIAL_CAPACITY);
 }
...
四种引用关系
强引用
软引用
（SoftReference的子类）：GC后内存不足的情
况将只有这种引用的对象回收
弱引用
WeakReference的子类）:GC时回收只有此引用
的对象（无论内存是否不足）
虚引用
没有特别的功能，类似一个追踪符，配合引用队
列来记录对象何时被回收。（实际上这四种引用
都可以配合引用队列使用，只要在构造方法中传
入需要关联的引用队列就行，在对象调用
finalize方法的时候会被写入到队列当中）
get
public T get() {
 Thread t = Thread.currentThread();
 ThreadLocalMap map = getMap(t);
 if (map != null) {
 ThreadLocalMap.Entry e = map.getEntry(this);
 if (e != null) {
 @SuppressWarnings("unchecked")
 T result = (T)e.value;
 return result;
 }
 }
 return setInitialValue();
}
（1）获取当前线程；
（2）获取当前线程的ThreadLocalMap；
（3）如果ThreadLocalMap非空，则将其对应的值返回；
（4）否则，初始化其对应的ThreadLocalMap，放入ThreadLocal
set
public void set(T value) {
 Thread t = Thread.currentThread();
 ThreadLocalMap map = getMap(t);
 if (map != null)
 map.set(this, value);
 else
 createMap(t, value);
 }
void createMap(Thread t, T firstValue) {
 t.threadLocals = new ThreadLocalMap(this, firstValue);
 }
ThreadLocalMap(ThreadLocal<?> firstKey, Object firstValue) {
 table = new Entry[INITIAL_CAPACITY];
 int i = firstKey.threadLocalHashCode & (INITIAL_CAPACITY - 1);
 table[i] = new Entry(firstKey, firstValue);
 size = 1;
 setThreshold(INITIAL_CAPACITY);
 }
装载因子阈值为2/3
注意区分 thredLocals、
ThreadLocal、ThreadLocalMap
threadLocals是Thread的属性
threadLocals对应的数据类型是ThreadLocalMap
ThreadLocalMap中key的类型为ThreadLocal
threadlocal是Thread的局部变量，别的线程无法进行访
问，因此可以实现多线程并发中的数据隔离
ThreadLocal是线程Thread中
属性threadLocals的管理者
代码
class Thread implements Runnable {
 /* ThreadLocal values pertaining to this thread. This map is maintained
 * by the ThreadLocal class. */
 ThreadLocal.ThreadLocalMap threadLocals = null;
 ...
每一个线程都有自己的ThreadLocalMap
实现细节
寻址
斐波那契散列法让数据更加散列，减少哈希碰撞采用0x61c88647这个值的原因是因为该值是斐波那契数也叫黄金分割数，hash值以此值为
增量的话，可以是的hash分布非常均匀，从而最大限度地去避免hash冲突
0x61c88647对应的十进制为1640531527。斐波那契散列的乘数可以
用(long) ((1L << 31) * (Math.sqrt(5) - 1))可以得到2654435769
拉链法寻址同一个下标位置发生冲突时，则+1 向后寻址，直到找到空位置或垃圾回收位置进行存储
ThreadLocalMap只有数组，并没有使用HashMap那样的链表通过拉链法
来处理Hash冲突，所以只能使用线性探测法
扩容原理扩容过程
首先把数组长度扩容到原来的 2 倍，oldLen * 2，实例化新数组
遍历 for，所有的旧数组中的元素，重新放到新数组中
在放置数组的过程中，如果发生哈希碰撞，则开放地址法顺延
同时还有检测 key 值的操作 if (k == null)，方便 GC
弱引用
如果Key使用强引用：当ThreadLocal的对象被回收了，但是ThreadLocalMap还持有ThreadLocal的强引用的话，如
果没有手动删除，ThreadLocal就不会被回收，会出现Entry的内存泄漏问题
如果Key使用弱引用：当ThreadLocal的对象被回收了，因为ThreadLocalMap持有ThreadLocal的弱引用，即使没有
手动删除，ThreadLocal也会被回收。value则在下一次ThreadLocalMap调用set,get,remove的时候会被清除
弱引用作为Entry的Key，可以多一层保障
弱引用ThreadLocal不会轻易内存泄漏，对应的
value在下一次ThreadLocalMap调用
set,get,remove的时候会被清除
过期key的清理流程
探测式清理（Proactive Cleanup）
探测式清理是指ThreadLocal在每次对ThreadLocal变量进行set或get操作时，
主动检查并清理那些已经不再使用的条目。这种清理方式是自动进行的，无需开
发者手动触发。
set/get两个方法都会间接调用expungeStaleEntries()方
法，该方法会遍历整个ThreadLocalMap，寻找并清除所
有的无效（即key为null）条目
启发式清理（Heuristic Cleanup）
启发式清理更多依赖于垃圾收集器的行为。当ThreadLocal对象被垃圾回收时，
其在ThreadLocalMap中的键值对的key会被设置为null，但值（Value）本身不
会被自动清理。因此，JDK使用启发式清理来间接触发清理过程。
试探的扫描一些单元格，寻找过期元素，也就是被垃圾回收的元素。当
添加新元素或删除另一个过时元素时，将调用此函数。它执行对数扫描
次数，作为不扫描（快速但保留垃圾）和与元素数量成比例的扫描次数
之间的平衡，这将找到所有垃圾，但会导致一些插入花费O（n）时间
会有两个地方调用到这个
cleanSomeSlots方法？？
set方法，在判断是否需要resize之前，会清理并rehash一遍
替换失效的节点时候，也会进行一次清理
原理总结
每个Thread维护着一个ThreadLocalMap的引用
ThreadLocalMap是ThreadLocal的内部类，用Entry来进行存储
调用ThreadLocal的set()方法时，实际上就是往ThreadLocalMap设置值，key是ThreadLocal对象，值是传递进来的对象
调用ThreadLocal的get()方法时，实际上就是在ThreadLocalMap获取值，key是ThreadLocal对象
坑
内存泄漏
原因
ThreadLocal变量被手动
设置为null后的引用链图
ThreadLocalMap使用ThreadLocal的弱引用作为key，当ThreadLocal变量被手动设置为null，即一个ThreadLocal没有外部强引用来引用
它，当系统GC时，ThreadLocal一定会被回收。这样的话，ThreadLocalMap中就会出现key为null的Entry，就没有办法访问这些key为
null的Entry的value，如果当前线程再迟迟不结束的话(比如线程池的核心线程)，这些key为null的Entry的value就会一直存在一条强引
用链：Thread变量 -> Thread对象 -> ThreaLocalMap -> Entry -> value -> Object 永远无法回收，造成内存泄漏。
Thread Ref–>Thread–>ThreadLocalMap–>Entry
内存泄漏的前提条件
ThreadLocal引用被设置为null，且后面没有set，get,remove操作
线程一直运行，不停止（线程池）
触发了垃圾回收。（Minor GC或Full GC）
解决
手动调用remove()
ThreadLocalMap的自动清除机制去清除过期Entry.（ThreadLocalMap的get(),set()时都会触发对过期Entry的清除,探测式清理）
线程重用导致信息混乱 Tomcat 的工作线程是基于线程池的
一旦线程重用，那么很可能首次从 ThreadLocal 获取
的值是之前其他用户的请求遗留的值。这时，
ThreadLocal 中的用户信息就是其他用户的信息。
在代码的 finally 代码块中，显式清除 ThreadLocal 中的数
据。这样一来，新的请求过来即使使用了之前的线程也不会
获取到错误的用户信息了
使用建议
每次使用完ThreadLocal，都调用它的remove()方法，清除数据
将ThreadLocal声明为private static，使它的生命周期与线程保持一致
ThreadLocal的应用场景
事务操作
pipeline操作保证在pipeline操作时，使用同一个连接批量传输数据
多数据源切换
存储Redis配置信息 redis本身虽然是单线程，但业务如果是多线程操作redis的话，就可能造成线程不安全
创建SimpleDataFormat对象 JDK8新引入的java.time包中的是类是不可变且线程安全的，可做SimpleDataFormat替代
QA
Entry的Key为什么要设计成弱引用呢？
避免内存泄漏
如果ThreadLocal的键是强引用（Strong Reference），那么只要线程还活着，ThreadLocal对象和线程局部变量的值就不会被垃圾回收
器回收，即使应用程序不再使用这个ThreadLocal对象。这是因为每个线程都持有自己的ThreadLocalMap，而ThreadLocalMap以强引用
形式持有ThreadLocal对象。这样一来，即使外部没有对ThreadLocal对象的引用，只要线程还在，就会导致内存泄漏
允许ThreadLocal对象被回收通过使用弱引用作为键，一旦外部对ThreadLocal对象的强引用消失，ThreadLocal对象就可以被垃圾回收器回收
自我清理机制 ThreadLocalMap中的条目（Entry）会在键（ThreadLocal对象）被垃圾回收后自动清理。虽然这个清理过程需要等到下一次访问
ThreadLocalMap时才会执行，但这种设计至少提供了一种机制，使得无用的线程局部变量最终能够被清除，从而减轻了内存泄漏的风险
threadlocal为什么存在扩容
ThreadLocal类型变量为何声明为static静态
节省内存当ThreadLocal变量被声明为静态时，它们属于类级别，而不是实例级别。这意味
着无论创建了多少个该类的实例，都只有一个ThreadLocal变量的实例。
生命周期管理将ThreadLocal变量声明为静态有助于管理其生命周期，确保只有一个全局实例
threadLocalHashCode 类似于 hashCode()，那
为什么 ThreadLocal 不重写 hashCode()？
public class ThreadLocal<T> {
 // 疑问 1：threadLocalHashCode 类似于 hashCode()，那为什么 ThreadLocal 不重写 hashCode()
 // ThreadLocal 的散列值，类似于重写 Object#hashCode()
 private final int threadLocalHashCode = nextHashCode();
 // 全局原子整型，每调用一次 nextHashCode() 累加一次
 private static AtomicInteger nextHashCode = new AtomicInteger();
 // 疑问：为什么 ThreadLocal 散列值的增量是 0x61c88647？ 斐波那契
 private static final int HASH_INCREMENT = 0x61c88647;
 private static int nextHashCode() {
 // 返回上一次 nextHashCode 的值，并累加 HASH_INCREMENT
 return nextHashCode.getAndAdd(HASH_INCREMENT);
 }
}
如果重写 Object#hashCode()，那么
threadLocalHashCode 散列值就会对所有散列
表生效。 threadLocalHashCode 散列值只在
ThreadLocal 内部的 ThreadLocalMap 使用。
HashMap类似
为什么不直接用线程id作为ThreadLocalMap的
key呢？
用了两个ThreadLocal成员变量的话。如果用线
程id作为ThreadLocalMap的key，怎么区分哪个
ThreadLocal成员变量呢？
public class TianLuoThreadLocalTest {
 private static final ThreadLocal<String> threadLocal1 = new ThreadLocal<>();
 private static final ThreadLocal<String> threadLocal2 = new ThreadLocal<>();
}
Entry的Key为什么要设计成弱引用呢？
如果Key使用强引用
当ThreadLocal的对象被回收了，但是ThreadLocalMap还持有
ThreadLocal的强引用的话，如果没有手动删除，ThreadLocal
就不会被回收，会出现Entry的内存泄漏问题
如果Key使用弱引用
当ThreadLocal的对象被回收了，因为ThreadLocalMap持有
ThreadLocal的弱引用，即使没有手动删除，ThreadLocal也会
被回收。value则在下一次ThreadLocalMap调用set,get，
remove的时候会被清除
使用弱引用作为Entry的Key，可以多一层保障：弱引用
ThreadLocal不会轻易内存泄漏，对应的value在下一次
ThreadLocalMap调用set,get,remove的时候会被清除
父子线程共享数据 InheritableThreadLocal
清理无效节点我理解，为什么要对有效节点再散列呢？ 线性探测只会遍历连续段，而清理无效节点会导致连续段产生断层。如果没有对有效节点做再散
列，那么有效节点在下次查询时就有可能探测不到了。
访问 ThreadLocal 一定会清理无效数据吗？
不一定。只有扩容会触发完整散列表清理，其他情况都不能保证清理，甚至不会触发。
自我清理机制：ThreadLocalMap中的条目（Entry）会在键（ThreadLocal对象）被垃圾回收后自动清理。虽然这个清理过程需要等到下一
次访问ThreadLocalMap时才会执行，但这种设计至少提供了一种机制，使得无用的线程局部变量最终能够被清除
thread是什么时候set threadLocal的值的？
threadLocal#set->Thread.ThreadLocal.ThreadLocalMap threadLocals = ThreadLocal中new ThreadLocalMap(this, firstValue);
过程 threadLocal#set方法判断ThreadLocalMap 是否为空，这个ThreadLocalMap 在Thread中定义：
ThreadLocal.ThreadLocalMap threadLocals
延伸
inheritableThreadLocals也是一样的 Thread中属性：ThreadLocal.ThreadLocalMap inheritableThreadLocals
只要是当前线程Thread parent = currentThread()，有threadLocal的地方，就会把自己放到当前Thread中去
解答
threadLocal#set，更严谨的说，是ThreadLocalMap 初始化的时候，
ThreadLocalMap 是在threadLocal中初始化，后Thread中的
ThreadLocal.ThreadLocalMap threadLocals 指向这个引用，相当于完
成了一个反转，thread中可以获取ThreadLocalMap threadLocals和
ThreadLocalMap inheritableThreadLocals
例
ThreadLocal<String> stringThreadLocal = new ThreadLocal<>();
InheritableThreadLocal<String> stringInheritableThreadLocal = new InheritableThreadLocal<>();
stringThreadLocal.set("ThreadLocal string");
stringInheritableThreadLocal.set("InheritableThreadLocal string");
Thread t = Thread.currentThread(); // t中可以看到stringThreadLocal 和 stringInheritableThreadLocal
为什么new Thread中获取不到threadLocal，而
可以获取到InheritableThreadLocal?
//主线程中赋值
ThreadLocal<String> stringThreadLocal = new ThreadLocal<>();
InheritableThreadLocal<String> stringInheritableThreadLocal = new
InheritableThreadLocal<>();
stringThreadLocal.set("ThreadLocal string");
stringInheritableThreadLocal.set("InheritableThreadLocal string");
Thread t = Thread.currentThread();
//子线程中分别打印两个变量的信息
new Thread(() -> {
 System.out.println(
 Thread.currentThread().getName() + " ThreadLocal value ：" +
stringThreadLocal.get());
 System.out.println(Thread.currentThread().getName() + "
InheritableThreadLocal value ："
 + stringInheritableThreadLocal.get());
}).start();
输出：
Thread-0 ThreadLocal value ：null
Thread-0 InheritableThreadLocal value ：InheritableThreadLocal string
因为在Thread#init方法中为
inheritableThreadLocals进行了赋值
this.inheritableThreadLocals =
ThreadLocal.createInheritedMap(parent.inhe
ritableThreadLocals);
ThreadLocalMap为什么要用ThreadLocal做
key，而不是用Thread做key？
假如使用Thread做key时，你的代码中定义了3个ThreadLocal对象，那么，通过Thread对象，它怎么知道要获取哪个ThreadLocal对象呢？因此，不能使用
Thread做key，而应该改成用ThreadLocal对象做key，这样才能通过具体ThreadLocal对象的get方法，轻松获取到你想要的ThreadLocal对象
ThreadLocalMap的enrty的
key为什么要设置成弱引用
将Entry的Key设置成弱引用，在配合线程池使用的情况下可能会有内存泄露的风险。设计成弱引用的目的是为了更好地对ThreadLocal进行回收，当我们在代码中将
ThreadLocal的强引用置为null后，这时候Entry中的ThreadLocal理应被回收了，但是如果Entry的key被设置成强引用则该ThreadLocal就不能被回收，这就是将其设置成弱引
用的目的
举例
三个线程中的每个线程的 ThreadLocalMap 的其中一
个 Entry 中的 key 使用的是同一个 ThreadLocal 类
型变量的地址。都指向了 ThreadLocal1
此时假设是强引用：多个线程依赖同一个 ThreadLocal1 ,此时线程 1 的 ThreadLocal1 使用结束了想要释放内
存，但是由于是强引用（因为还有其他线程还在指向 ThreadLocal1）,这就导致了线程1 的持有 ThreadLocal1 的
Entry 占有的内存无法释放，导致了内存泄露，使用弱引用的时候，这种问题就可以解决
ThreadLocal真的会导致内存泄露？
当线程池创建一个线程，执行线程的过程中创建了ThreadLocal对象实例，并保存了value。当线
程执行完毕之后并没有被销毁，而是存储保存在了线程池之中，此时ThreadLocal对象与value都
依旧存在着，当线程从线程池中拿出再次运行，在执行过程中又创建了新的ThreadLocal对象，
那么之前的ThreadLocal对象没有强引用指向，被gc回收，之前保存的value无法被访问到，但依
旧有ThreadLocalMap的强引用指向，不会被回收，造成了内存泄露
每个线程都有一个ThreadLocalMap，这个Map可以存放多个
ThreadLocal变量。当ThreadLocal变量没有被移除时，它所引
用的对象也会一直存放在线程的ThreadLocalMap中，这会导
致ThreadLocalMap变得很大，从而占用大量的内存空间，最
终导致内存泄漏
ThreadLocalMap怎么解决Hash冲突的？ 开放定址法
ThreadLocal是如何定位数据的？
1. 通过key的hashCode取余计算出一个下标 key.threadLocalHashCode & (len-1);
2. 通过下标，在数组中定位具体Entry，如果key正好是我们所需要的key，说明找到了，则直接返回数据
3. 如果第2步没有找到我们想要的数据，则从数组的下标位置，继续往后面找
4. 如果第3步中找key的正好是我们所需要的key，说明找到了，则直接返回数据
5. 如果还是没有找到数据，再继续往后面找。如果找到最后一个位置，还是没有找到数据，则再从
头，即下标为0的位置，继续从前往后找数据
6. 直到找到第一个Entry为空为止
什么情况下，即使ThreadLocal不remove，也不
会内存泄漏？
短生命周期的线程
对于那些生命周期较短的线程（即它们执行完任务就结束了），即使不调用ThreadLocal的remove()方法，当线程结束并且没有其他引用
指向这个线程对象时，线程对象将被垃圾回收。这也意味着线程对象持有的ThreadLocalMap和其中存储的所有条目（包括ThreadLocal变
量及其对应的值）也都会被垃圾回收，前提是没有其他地方持有这些ThreadLocal变量的强引用
单次使用的ThreadLocal变量
如果ThreadLocal变量只在一个短生命周期的线程中使用一次，并且之后没有任何强引用指向这个ThreadLocal变量（即它们可以
被垃圾回收），那么即使这个变量的remove()方法没有被显式调用，ThreadLocal变量及其存储的值也不会导致内存泄漏，因为
当ThreadLocal实例本身被垃圾回收时，其在ThreadLocalMap中的条目就会自动清理
InheritableThreadLocal
是什么借助于inheritableThreadLocals，可以实现，创建线程向被创建线程数据传递
目的父线程和子线程之间共享数据
应用安例
ThreadLocal<String> stringThreadLocal = new ThreadLocal<>();
InheritableThreadLocal<String> stringInheritableThreadLocal = new InheritableThreadLocal<>();
stringThreadLocal.set("ThreadLocal string");
stringInheritableThreadLocal.set("InheritableThreadLocal string");
//子线程中分别打印两个变量的信息
new Thread(() -> {
 System.out.println(
 Thread.currentThread().getName() + " ThreadLocal value ：" + stringThreadLocal.get());
 System.out.println(Thread.currentThread().getName() + " InheritableThreadLocal value ："
 + stringInheritableThreadLocal.get());
}).start();
结果
Thread-0 ThreadLocal value ：null
Thread-0 InheritableThreadLocal value ：
InheritableThreadLocal string
在子线程中，可以获取到父线程的
InheritableThreadLocal类型变量的值，而不
能获取到ThreadLocal类型变量的值
原理
代码
public class Thread implements Runnable {
 ThreadLocal.ThreadLocalMap threadLocals = null;
 ThreadLocal.ThreadLocalMap inheritableThreadLocals = null;
 private void init(...,boolean inheritThreadLocals) {
 Thread parent = currentThread();
 if (inheritThreadLocals && parent.inheritableThreadLocals != null)
 // 将父线程的inheritableThreadLocals赋值给子线程的inheritableThreadLocals
 this.inheritableThreadLocals =
 ThreadLocal.createInheritedMap(parent.inheritableThreadLocals);
...
如果parent的inheritableThreadLocals不是
null，那么就会将当前线程的
inheritableThreadLocals设置为parent的
inheritableThreadLocals
parent是什么？就是创建这个线程
的线程，也就是平时说的父线程
public class InheritableThreadLocal<T> extends ThreadLocal<T> {
 protected T childValue(T parentValue) {
 return parentValue;
 }
 ThreadLocalMap getMap(Thread t) {
 return t.inheritableThreadLocals;
 }
 void createMap(Thread t, T firstValue) {
 t.inheritableThreadLocals = new ThreadLocalMap(this, firstValue);
 }
}
通过对getMap和createMap方法的重写，可以
看得出来，只是内部操作的变量发生了变化，从
threadLocals转换为inheritableThreadLocals
描述
InheritableThreadLocal的核心概念仍旧是ThreadLocal.ThreadLocalMap以及ThreadLocal
InheritableThreadLocal是 ThreadLocal的子类
在Thread内部通过维护 ThreadLocal.ThreadLocalMap
inheritableThreadLocals 进行父子线程数据的传递
这个数据则是通过在创建Thread对象的时候，借助于内部的init方法，调用
createInheritedMap方法，从父线程（当前创建线程）中复制的一份
过程
当创建一个新的线程的时候X，X线程就会有 ThreadLocalMap 类型的 inheritableThreadLocals ，因为它是 Thread 类的一个属性
先得到当前线程存储的这些值，例如 Entry[] parentTable = parentMap.table; 。再通过一个 for 循环，不断的把当前线程的这些
值复制到我们新创建的线程X 的inheritableThreadLocals 中
结果就是我们创建的新线程X 的inheritableThreadLocals 变量中已经有了值了
和threadlocal关系
Thread内部的inheritableThreadLocals变量，与内部的那个threadLocals没有什么关系
换个思路理解，他就是另外一个ThreadLocal，假定Thread定义了两个“ThreadLocal“，其中一个在创建线程的时候会从父线程复制一份
只不过从实现上、以及逻辑上，他继承了ThreadLocal而已，然后覆盖了几个方法
注意
InheritableThreadLocal 区域在拷贝
后依然是线程隔离的
在完成拷贝后，父子线程对 InheritableThreadLocal 的操作依然是相互独立的。子线程对
InheritableThreadLocal 的写不会影响父线程的 InheritableThreadLocal，反之亦然
拷贝过程在父线程执行虽然拷贝数据的代码写在子线程的构造方法中，但是依然是在父线程执行的。子线程是在调用 start() 后才开始执行的
线程池的影响使用 InheritableThreadLocal，那么线程池中的线程拷贝的数据来自于第一个提交任务的外部线程，即后面的外部
线程向线程池中提交任务时，子线程访问的本地变量都来源于第一个外部线程，造成线程本地变量混乱
QA
InheritableThreadLocal什么时候
复制的父线程的值？
在子线程创建的时候，将父线程的本地变量拷贝
到子线程中值传递
为什么是值传递源码中未进行拷贝，直接返回父线程对象的引用
什么时候做的赋值 InheritableThreadLocal是在new Thread中的init()方法给赋值的即线程创建的时候，将父线程的inheritableThreadLocals的值赋给子线程的inheritablesThreadLocals对象
TransmittableThreadLocal
是什么 TransmittableThreadLocal（简称TTL）是alibaba提供的一个工具包中的类，主要作用就是解决线程池场景下的变量传递问题。继承自InheritableThreadLocal
解决问题解决了InheritableThreadLocal只能再new Thread的时候传递本地变量，无法在使用线程池等会复用线程的场景下，变量值传递的问题
用途
链路追踪，如日志链路追踪（比如traceId在线程池中传递）
用户会话信息
中间件线程池信息传递。如Hystrix（不过hystrix自己实现了类似的一套）
TTL使用方式（两种）
用TTL代码包裹让他生效
// 用TransmittableThreadLocal
TransmittableThreadLocal<Integer> ttl = new TransmittableThreadLocal<>();
// 1. 用TtlCallable.get(callable)包裹callable 或用TtlRunnable.get(callable)包裹runnable
Runnable runnable = () -> System.out.println("ttl信息：" + ttl.get());
TtlRunnable.get(runnable);
// 2. 用TtlExecutors.getTtlExecutorService包裹
ExecutorService ttlExecutorService = TtlExecutors.getTtlExecutorService(Executors.newFixedThreadPool(1));
ttlExecutorService.execute(() -> System.out.println("ttl信息：" + ttl.get()));
用JavaAgent自动修改字节码启动jar的时候，附加上参数-javaagent:/xx/transmittable-thread-local.jar
/opt/soft/jdk/jdk1.8.0_191/bin/java
 -javaagent:/xx/transmittable-thread-local.jar
 -jar /opt/xxx.jar
设计模式装饰器模式去增强Runnable等任务
QA
TTL为什么不直接继承ThreadLocal？ 因为有些业务需要用到ITL特性，如果直接继承ThreadLocal，就会丢失ITL的父拷贝到子线程数据的特性(子线程创建时拷贝)
为什么需要在run执行完之后调用restore()？
restore里面会主动调用remove()回收，避免内存泄露（会删除子线程新增的TTL）
不调用restore()的话，就会覆盖之前backup备份部分子线程的数据，这样可能在业务上有隐患
TTL存在线程安全问题？ 存在的，因为默认都是引用类型拷贝，如果子线程修改了数据，主线程是可以感知到的
TTL是否存在内存泄露问题？ TTL维护的holder本身是一个static来的，使用的时候会调用restore(),然后里面显式调用remove()清楚子线程新增TTL，所以正确使用下是没有内存泄露问题的
TODO TransmittableThreadLocal后面再总结
LockSupport
LockSupport.park
通过UnSafe类使用了CPU的原语。在AQS中使用park的主要作用是，让排队的线程阻塞掉（停止其自
旋，自旋会消耗CPU资源），并在需要的时候，可以方便的唤醒阻塞掉的线程
LockSupport.park()后，通过jstack查看，线程进入到WAITING,这里的状态是线程在JVM中的线程
状态，通过ps命令查看本进程中的操作系统线程状态，线程的状态是Sleep
LockSupport.unpark 解除线程的WAITING/Sleep
QA
如何解除线程的WAITING/Sleep，让线
程继续运行呢？
第一种方式通过LockSupport.unpark(thread)
第二种方式通过thread.interrupt()
wait,notify,notifyAll和park,unpark的区别
wait，notify 和 notifyAll 必须配合 Object Monitor 一起使用，而 park，unpark 不必
park & unpark 是以线程为单位来【阻塞】和【唤醒】线程，而 notify 只能随机唤醒一个等待线程，notifyAll 是唤醒所有等待线程，就不那么【精确】
park & unpark 可以先 unpark，而 wait & notify 不能先 notify
4. 线程
线程状态切换关键字
分类
wait 让出CPU资源和锁资源
wait是object方法
使当前线程暂停执行并释放对象锁标志，让其他线程可以进入Synchronized数据块，当前线程被放入对象等待池中
sleep
让出CPU资源，但是不会释放锁资源
sleep是线程方法
虽然Thread.sleep()会让出CPU，但它不会释放已经持有的任何监视器锁（synchronized locks）
sleep()方法给其他线程运行机会时不考虑其他线程的优先级，因此会给低优先级的线程运行的机会
yeild 让出CPU调度，Thread类的方法
提示调度程序，当前线程愿意放弃当前对处理器的使用。这时，当前线程将会被置为就绪状态，和其他线
程一样等待调度，这时候根据不同优先级决定的概率，当前线程完全有可能再次抢到处理器资源
执行yield()方法后转入就绪（ready）状态
join
一种特殊的wait
join底层是用wait实现的，使用了并发设计模式中的保护性暂停模式。保护性暂停
是一种等待唤醒机制的一种规范 ,也可以理解为使用中设计模式
如 join,future
主线程等待子线程的终止主线程的代码块中，如果碰到了t.join()方法，此时主线程需要等待（阻
塞），等待子线程结束了,才能继续执行t.join()之后的代码块
t.join();//主要用于等待t线程运行结束，若无此句，main则会执行完毕，导致结果不可预测
join方法通常用于以下情况
确保并发执行的线程在继续执行之前完成特定的任务。
在多线程计算中收集结果数据。
确保资源清理或者应用逻辑的正确顺序
join()方法的死锁 Thread.currentThread().join(); 如果调用了线程自身调用join()这个方法，那么线程一直在阻塞，
无法终止。因为它自己在等待自己结束；这无疑会造成死锁!
notify
当调用 notify()方法后，将从对象的等待池中移走一个任意的线程并放到锁标志等待池中，只有锁标志等待池中的线程能够获取锁标志
唤醒在此对象监视器上等待的单个线程，选择是任意性的
notifyAll
从对象等待池中移走所有等待那个对象的线程并放到锁标志等待池中
唤醒在此对象监视器上等待的所有线程
interrupt
用于指示一个线程应该停止正在执行的任务。这个机制允许一个线程告诉另一个线程，它应该尽快终止正在进行的操作。interrupt机制是
一种协作机制。当一个线程打断另一个线程时，被打断的线程不会立即停止执行。相反，它会收到一个打断信号（interrupt signal），这
个信号告诉它有人希望它停止正在做的事情。接收到打断信号的线程如何响应完全取决于该线程
wait 和notify的原理
步骤
调用wait() 首先会获取监视器锁，获得成功后，会让线程进入等待状态进入等待队列并且释放锁
然后当其他线程调用notify或者notifyall以后，会选择从等待队列中唤醒任意一个线程
而执行完notify方法以后，并不会立马唤醒线程，原因是当前线程仍然持有这把锁，处于等待状态的线程无法获得锁。必须
要等到当前的线程执行完按monitorexit指令之后，也就是被释放之后，处于等待队列的线程就可以开始竞争锁了
wait和notify为什么要放在synchronized里面
wait方法的语义有两个
释放当前的对象锁
使得当前线程进入阻塞队列
而这些操作都和监视器是相关
的，所以wait必须要获得一个
监视器锁
notify也一样，它是唤醒一个线程，所以需要知道待唤醒的线程在哪里，就必须找到这个对
象获取这个对象的锁然后去到这个对象的等待队列去唤醒一个线程
sleep与Wait的区别
sleep是线程方法，wait是object方法
wait，notify,notifyall 都是Object对象的方法，是一起使用的，用于锁机制；sleep是线程的方法，和锁无关。
所属类
wait()方法是Object类的方法
sleep()方法是Thread类的静态方法，它通常被用来暂停当前正在执行的线程
锁的处理
当线程调用一个对象的wait()方法时，它必须持有该对象的监视器锁。调用wait()会导致当前线
程释放锁，并进入该对象的等待池，直到其他线程调用同一个对象的notify()或notifyAll()方法
sleep()方法不会释放任何锁，直到睡眠时间结束或被中断
用途
wait()方法通常用于线程间通信
sleep()方法主要用于暂停当前线程一段指定的时间
异常处理
wait()方法可以在任何同步块或同步方法中被调用。如果线程在等待期间被中断，wait()方法会抛出InterruptedException
sleep()方法调用时，当前线程不需要持有任何对象锁。如果线程在睡眠期间被中断，sleep()方法同样会抛出InterruptedException
唤醒条件
notify()或notifyAll()被唤醒，或者在wait(long timeout)指定的等待时间结束后自动唤醒
sleep()方法的睡眠时间结束后，线程会自动变为可运行状态。也可以通过其他线程调用interrupt()方法提前唤醒
sleep和yield的不同之处
sleep(long)方法会使线程转入超时等待状态，时间到了之后才会转入就绪状态；
yield()方法不会将线程转入等待，而是强制线程进入就绪状态
使用sleep(long)方法需要处理异常，而yield()不用
线程状态
进程状态
进程一般状态
分类
运行态（Running）
就绪态（等待态）（Waiting）
当进程被分配到除 CPU 以外的所有必要资源后，就处于就绪状态
只要再获得 CPU 的使用权，就可以里立即运行
其它资源都准备好，只差 CPU 资源的状态，就是就绪状态其它资源包括进程控制块(PCB)、内存、栈空间、堆空间等
阻塞态（Blocked）
进程由于某种原因，如：其它设备未就绪而
无法继续执行，从而放弃 CPU 的状态
比如说有个进程要请求打印机，但是由于打印机正处于被占
用的情况，进程收不到回应，那么就会处于阻塞状态
状态转换图
三种状态之间的转换
运行—>就绪
主要是进程占用CPU的时间过长，而系统分配给该进程占用CPU的时间是有限的；
在采用抢先式优先级调度算法的系统中,当有更高优先级的进程要运行时，该进程就被迫让出CPU，该进程便由执行状态转变为就绪状态。
就绪—>运行运行的进程的时间片用完，调度就转到就绪队列中选择合适的进程分配CPU
运行—>阻塞正在执行的进程因发生某等待事件而无法执行，则进程由执行状态变为阻塞状态，如发生了I/O请求
阻塞—>就绪进程所等待的事件已经发生，就进入就绪队列
不可能发生的状态转换
阻塞—>运行即使给阻塞进程分配CPU，也无法执行，操作系统在进行调度时不会从阻塞队列进行挑选，而是从就绪队列中选取
就绪—>阻塞就绪态根本就没有执行，谈不上进入阻塞态
linux下的进程进程状态
状态转换图
分类
R运行状态（running）并不意味着进程一定在运行中，它表明进程要么是在运行中要么在运行队列里
S睡眠状态（sleeping) 意味着进程在等待事件完成，也叫做可中断睡眠（interruptible sleep）
D磁盘休眠状态（Disk sleep）也叫不可中断睡眠状态（uninterruptible sleep），在这个状态的进程通常会等待IO的结束
T停止状态（stopped）
可以通过发送 SIGSTOP 信号给进程来停止（T）进程。这个被暂停的进程可以通过发送
SIGCONT 信号让进程继续运行
X死亡状态（dead）这个状态只是一个返回状态，你不会在任务列表里看到这个状态
线程状态
状态分类
新建状态（New）
就绪状态（Runnable）
线程对象创建后，其它线程调用了该对象的start方法，该状态的线程会位于可
运行的线程池中，变得可运行，等待cpu的使用权
运行状态（Running）就绪状态的线程获取到了cpu的时间片，执行程序代码
阻塞状态（Blocked）
阻塞状态是线程因为某种原因放弃cpu的使用
权，暂停或停止运行，直到线程进入就绪状态，
才有机会获得cpu的青睐从而转入运行状态
阻塞状态又分为三种
等待阻塞
运行的线程执行wait方法，则该线程会释放占用的的所有资
源，jvm会把该线程放入 “等待池” 中，进入这个状态后是不能
被自动唤醒的，需要调用notify/notifyAll 方法才能被唤醒
同步阻塞
运行的线程在获取对象的同步锁时，若该同步锁被其它线程占
用，则jvm会把该线程放入 “锁池” 中
其它阻塞
运行的线程执行sleep/join方法、或者发出了I/O请求时，jvm会
把该线程置为阻塞状态。当sleep方法等待超时、join方法等待
线程终止或者超时、I/O处理完毕时，线程将重新转入就绪状态
死亡状态（Dead）线程执行完了或者因为异常而退出的run方法，该线程生命周期结束
线程的状态可能会受到操作系统调度策略的影响，因此不同的操作系统可能会有不同的状态定义
Java线程状态
状态之间的转换
状态分类
新建状态(NEW) 用new关键字新建一个线程，这个线程就处于新建状态
运行状态(RUNNABLE)
分为2个子状态
就绪状态（READY)
当线程对象调用了start()方法之后，线程处于就绪状态，就绪意味着该
线程可以执行，但具体啥时候执行将取决于JVM里线程调度器的调度
不允许对一个线程多次使用start。
线程执行完成之后，不能试图用start将其唤醒
其他状态 ->就绪
线程调用start()，新建状态转化为就绪状态。
线程sleep(long)时间到，等待状态转化为就绪状态。
阻塞式IO操作结果返回，线程变为就绪状态。
其他线程调用join()方法，结束之后转化为就绪状态。
线程对象拿到对象锁之后，也会进入就绪状态。
运行状态(RUNNING)
处于就绪状态的线程获得了CPU之后，真正开始执行run()方法的线程执行体时，意味着该线程就已经处于运行状态
对于单处理器，一个时刻只能有一个线程处于运行状态
运行状态转变为就绪状态
线程失去处理器资源。线程不一定完整执行的，执行到一半，说
不定就被别的线程抢走了。
调用yield()静态方法，暂时暂停当前线程，让系统的线程调度器
重新调度一次，它自己完全有可能再次运行。
操作系统中的就绪和运行两种状态，在Java中统称为RUNNABLE
阻塞状态(BLOCKED)
阻塞状态表示线程正等待监视器锁，而陷入的状态
以下场景线程将会阻塞
线程等待进入synchronized同步方法
线程等待进入synchronized同步代码块
等待状态(WAITING)
进入该状态表示当前线程需要等待其他线程做出一些的特定的动作（通知或中断）
运行->等待
当前线程运行过程中，其他线程调用join方法，当前线程将会进入等待状态
当前线程对象调用wait()方法。 LockSupport.park()：出于线程调度的目的禁用当前线程
等待->就绪
等待的线程被其他线程对象唤醒，notify()和notifyAll()
LockSupport.unpark(Thread)，与上面park方法对应，给出许可证，解除等待状态
超时等待状态(TIMED_WAITING)
区别于WAITING，它可以在指定的时间自行返回
运行->超时等待
调用静态方法，Thread.sleep(long)
线程对象调用wait(long)方法
其他线程调用指定时间的join(long)
LockSupport.parkNanos()
LockSupport.parkUntil()
消亡状态(TERMINATED) 线程的终止，表示线程已经执行完毕
run()和call()线程执行体中顺利执行完毕，线程正常终止。
线程抛出一个没有捕获的Exception或Error
线程间的通信机制都有哪些
共享内存
线程或进程通过读写同一块内存区域来进行通信
线程共享进程的堆内存，可以通过修改对象字段或使用并发集合（如 ConcurrentHashMap）实现共享内存通信
消息传递
线程或进程通过发送消息来交换信息
BlockingQueue 提供了一个线程安全的队列实现，用于在生产者和消费者之间传递消息
管道
java
字节流：PipedOutputStream 和 PipedInputStream
字符流：PipedReader 和 PipedWriter
操作系统实现 UNIX 和 Linux 系统提供管道机制，允许进程间通过 pipe 系统调用进行通信
信号量
使用计数器控制对共享资源的访问
Semaphore
等待/通知机制通过 Object 类的 wait()、notify() 和 notifyAll() 方法实现
原子操作 AtomicInteger
锁 ReentrantLock、ReadWriteLock、synchronized
事件线程可以等待一个或多个事件的发生
同步辅助工具
CountDownLatch、CyclicBarrier、Phaser
join
ThreadLocal
异步编程和结果合并 CompletableFuture
并发集合 ConcurrentHashMap、BlockingQueue
线程间如何同步变量
synchronized synchronized 实际上是对访问修改共享变量的代码块进行加互斥锁，多个线程对synchronized代码块的访问时，某一时刻仅仅有一个线程在访问和
修改代码块中的内容（加锁），其他所有的线程等待该线程离开代码块时（释放锁）才有机会进入synchronized代码块
Lock 接口
ReentrantLock Condition 接口与 Lock 配合使用
ReadWriteLock
wait 和 notify
Atomic 类
AtomicInteger
AtomicLong
LongAdder
并发工具类
CountDownLatch、Semaphore
CyclicBarrier 和 Phaser 用于多线程间的同步点，允许一组线程相互等待，直到达到某个公共屏障
Thread.join() 用于在一个线程中等待另一个线程完成
阻塞队列 LinkedBlockingQueue 队列是先进先出的顺序（FIFO）
创建线程方式
继承Thread类
实现Runnable接口，覆盖run方法
实现Callable接口需要借助FutureTask类，比如获取返回结果
使用线程池
线程中断
Thread.interrupt() 中断线程这里的中断线程并不会立即停止线程，而是设置线程的中断状态为 true（默认是 flase）
Thread.currentThread().isInterrupted() 测试当前线程是否被中断线程的中断状态会受这个方法的影响，调用一次可以使线程中断状态变为 true，调用两次会使这个线程的中断状态重新转为 false
Thread.isInterrupted() 测试当前线程是否被中断与上面方法不同的是调用这个方法并不会影响线程的中断状态
QA
主线程如何捕获子线程抛出的异常？
线程设计的理念：“线程的问题应该线程自己本身来解决，而不要委托到外部。”
UncaughtExceptionHandler
给某个thread设置一个UncaughtExceptionHandler，可以确保在该线程
出现异常时能通过回调UncaughtExceptionHandler接口的public void
uncaughtException(Thread t, Throwable e) 方法来处理异常
虽然是在回调方法中处理异常，但这个回调方法在执行时依然还
在抛出异常的这个线程中！
如果线程是通过线程池创建，线程异常发生时
UncaughtExceptionHandler接口不一定会立即回调
让主线程等待子任务执行的各种方式
1. 利用java多线程工具Future.get()获取数据
2. 利用CountDownLatch让主线程等待子线程任务结束
3. 利用CyclicBarrier让主线程等待子线程
4. 利用CompletionService
5. thread.join()
6. CompletableFuture
Java 线程和操作系统的线程有啥区别？
Java线程
抽象概念 Java 线程是 JVM 中的一个抽象概念，代表独立的执行流，便于在 Java 程序中进行多线程编程
跨平台性 Java 线程的行为在所有支持 Java 的平台上保持一致，无论底层操作系统是什么，都使用相同的 API 和行为模式
轻量级 Java 线程被认为是轻量级的，因为它们在 JVM 层面被管理，不直接拥有操作系统层面的资源，如独立的内存空间或文件描述符
JVM 管理 Java 线程的创建、调度和管理由 JVM 负责，包括线程的生命周期管理、同步机制和内存管理等
协作式调度 Java 线程采用协作式调度机制，依赖于 JVM 的调度策略来管理线程间的切换，而不是由操作系统的内核直接抢占
操作系统线程
内核对象操作系统线程是操作系统内核中的基本执行单元，每个线程都是内核中的一个对象
内核管理操作系统线程的创建、调度和资源分配由操作系统的内核负责，包括线程间同步和通信机制
重量级与 Java 线程相比，操作系统线程是重量级的，因为每个线程都拥有自己的系统资源，例如独立的内存空间和文件描述符
平台相关性操作系统线程的具体实现和性能特性依赖于所在的操作系统，不同操作系统间可能存在显著差异
为什么wait和notify方法要在同步块中调用？
当线程执行对象的同步代码块时，它会自动获取该对象的监视器锁。wait() 和 notify() 方法依赖于
监视器锁来提供线程间的通信机制
如果在非同步块中调用 wait() 或 notify() 方法，会抛出 IllegalMonitorStateException。这是因
为任何线程想要调用对象上的 wait()、notify() 或 notifyAll() 方法，必须是该对象监视器的所有者
为什么获取监视器锁要在同步代码块中？
保证操作的原子性防止了数据的不一致性和竞争条件
避免死锁和保持顺序如果没有强制要求在同步代码块中调用 wait() 和 notify()，程序很容易陷入死锁。比如，一个线程在尝试等待其他线程通知之前没有正确释放锁，可能
导致其他线程无法进入同步代码块来发出通知。通过在同步代码块中调用这些方法，Java 确保了锁的正确获取和释放，以及线程间通信的有序性
确保可见性和顺序使用同步代码块确保了内存可见性，当进入同步代码块时，确保了对共享变量的更改对其他线程可见，反之亦然。这是因为同步块在进入时会清空本地内
存中的共享变量值，在退出时会将更改写回主内存。这对于 wait() 和 notify() 来说尤其重要，因为它们通常用于控制对这些共享变量的访问
有三个线程T1 T2 T3，如何保证他们按顺序执行
join
Thread t1 = new Thread(() -> {System.out.println("Thread T1 is running.");});
Thread t2 = new Thread(() -> {System.out.println("Thread T2 is running.");});
Thread t3 = new Thread(() -> {ystem.out.println("Thread T3 is running.");});
try {
 t1.start(); t1.join(); // 等待t1执行完毕
 t2.start(); t2.join(); // 等待t2执行完毕
 t3.start(); t3.join(); // 等待t3执行完毕
} catch //....
CompletableFuture
CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
 System.out.println("Thread T1 is running.");
}).thenRun(() -> { System.out.println("Thread T2 is running.");
}).thenRun(() -> { System.out.println("Thread T3 is running.");});
future.join(); // 等待所有任务完成，没有这一句也是一样的结果输出
newSingleThreadExecutor
ExecutorService executor = Executors.newSingleThreadExecutor();
executor.submit(() -> System.out.println("Thread T1 is running."));
executor.submit(() -> System.out.println("Thread T2 is running."));
executor.submit(() -> System.out.println("Thread T3 is running."));
executor.shutdown();
wait()和notify()，CountDownLatch，Lock和Condition方法也可以实现，但是写起来太麻烦了
一个线程运行时发生异常会怎样？
异常被捕获如果异常在线程的run方法内部被捕获，那么线程可以决定如何处理这个异常，可以选择记录日志、重试失败的操作、清理资源或者通过某种方式恢复状态
异常未被捕获，则会
有以下几个步骤发生
线程终止
调用UncaughtExceptionHandler
如果为该线程或线程的ThreadGroup设置了UncaughtExceptionHandler，则会调用该处理程序的uncaughtException方法来处理异常
没有设置UncaughtExceptionHandler，则默认行为是将异常的堆栈跟踪打印到System.err
对其他线程基本无影响一个线程的异常终止通常不会直接影响其他线程。其他线程可能会因为线程间的协作机制（如join、wait/notify等）间接受到影响
Thread.interrupt() 方法不会立即停止目标线
程，那还有什么用？
协作性与立即终止线程的方法（如不安全的 Thread.stop()，已被废弃）相比，interrupt() 方法允许线程按照自己的逻辑和速度来响应中断请求。这种方式更加安全
和灵活，因为它允许线程完成重要的清理工作，避免了资源泄露和状态不一致的风险
灵活性线程可以根据当前执行的上下文来决定如何响应中断。例如，如果线程正在执行一项可中断的任务，它可以立即停止任务；如果正在执行不可中断的关键操作，
它可以等到操作完成后再停止
资源清理中断机制允许线程在终止前执行清理操作，如关闭文件句柄、释放锁、通知其他线程等。这有助于保持系统资源的正常管理和应用状态的一致性
避免阻塞当线程因调用 sleep()、wait() 或 join() 等方法而阻塞时，interrupt() 可以打断这种阻塞状态，抛出 InterruptedException，从而允许线程退出阻塞状态并
适当地响应中断
增强可控性通过检查中断状态（使用 Thread.isInterrupted() 或 Thread.interrupted()），线程可以在执行长时间循环或重复任务时，定期检查是否有中断请求，从而更灵
活地控制执行流程
5. 线程池
submit和execute区别
相同 submit方法比execute方法多的只是将提交的任务（不管是runnable类型还是callable类型）包装成RunnableFuture，其核心逻辑还是调用的execute方法执行。
不同
接收的参数不一样
execute Runnable
submit 可以是Runnable,也可以是Callable
返回值
execute execute没有返回值
submit submit有返回值,返回值是Future
Exception处理 submit方便Exception处理
分类
newFixedThreadPool
是什么
适用场景
当有固定数量的并发任务需要处理时
希望限制线程数量，防止过多线程导致资源耗尽
注意事项
由于线程数量是固定的，如果同时有大量任务提交，可能会导致工作队列积压，因此需要根据实际情况来选择线程池的大小
固定线程池会创建一个固定数量的线程，并在池中保持这些线程的数量不变
当任务量较小时，固定线程池可能会浪费一些线程资源，如果在任务量不确定的情况下，可以考虑使用缓存线程池
newCachedThreadPool
是什么缓存线程池会根据需要动态地创建线程，并在一段时间内重用空闲线程，通常是60秒
适用场景缓存线程池适用于短期任务数量不确定的场景，特别是在处理大量的短期任务时，它可以根据任务的数量动态地分配和回收线程，以提供最佳的并发性能
注意事项
虽然缓存线程池在短期任务处理上非常高效，但在处理长期任务时，可能会导致过多的线程占用系统资源
当不再需要线程池时，务必调用 shutdown() 方法来释放线程资源，避免资源泄漏
newSingleThreadExecutor
是什么单线程池只有一个工作线程，用于按照顺序执行提交的任务，所有任务按照它们的提交顺序在同一个线程中执行
适用场景
单线程池适用于需要按顺序执行任务的场景，例如需要保证任务的顺序执行，避免并发带来的问题
也适用于需要在单个线程中执行任务，以避免并发的复杂性
注意事项
单线程池适用于保证任务顺序执行的场景，但不适合需要高并发处理的情况
虽然只有一个线程在执行任务，但如果有异常抛出，线程池会创建一个新线程来继续执行后续任务
newScheduleThreadPool
是什么调度线程池（ScheduledThreadPool）是 Java 中一种特殊的线程池类型，用于执行定时任务或周期性任务
适用场景调度线程池适用于需要按计划执行任务的场景，例如：定时生成报表、定时数据清理、周期性检查等
注意事项
调度线程池适用于需要按计划执行任务的场景，但需要考虑任务执行时间和调度间隔，避免任务堆积或执行过长导致的问题
当不再需要线程池时，务必调用 shutdown() 方法来释放线程资源，避免资源泄漏
newSingleThreadScheduledExecutor 单例的定时线程池
newWorkStealingPool
是什么
ForkJoinPool任务窃取线程池
Java8新添加的线程池
ForkJoinPool的好处是，把1个任务拆分成多个“小任务”，把这些“小任务”分发到多个线程上执行。这些“小任务”都执行完成后，再将结果合并
之前的线程池中，多个线程共有一个阻塞队列，而newWorkStealingPool 中每一个线程都有一个自己的队列
当线程发现自己的队列没有任务了，就会到别的线程的队列里获取任务执行。可以简单理解为”窃取“
适用场景
注意事项
参数
corePoolSize
在刚刚创建ThreadPoolExecutor的时候，线程并不会立即启动，而是要等到有任务提交时才会启动，除非调用了
prestartCoreThread/prestartAllCoreThreads事先启动核心线程
考虑到keepAliveTime和allowCoreThreadTimeOut超时参数的影响，所以没有任务需要执行的时候，线程池的大小不一定是corePoolSize
阻塞队列如果当前大小已经达到了corePoolSize 大小，就将新提交的任务提交到阻塞队列排队，等候处理workQueue.offer(command)
maximumPoolSize
如果队列容量已达上限，并且当前大小poolSize没有达到maximumPoolSize，那么就新增线程来处理任务
如果队列已满，并且当前线程数目也已经达到上限，那么意味着线程池的处理能力已经达到了极限，此时需要拒绝新增加的任
务。至于如何拒绝处理新增的任务，取决于线程池的饱和策略RejectedExecutionHandler
allowCoreThreadTimeOut 该属性用来控制是否允许核心线程超时退出。If false,core threads stay alive even when idle.If true, core threads use
keepAliveTime to time out waiting for work
keepAliveTime
如果一个线程处在空闲状态的时间超过了该属性值，就会因为超时而退出。举个例子，如果线程池的核心大小corePoolSize=5，而当前大小
poolSize =8，那么超出核心大小的线程，会按照keepAliveTime的值判断是否会超时退出。如果线程池的核心大小corePoolSize=5，而当前大小
poolSize =5，那么线程池中所有线程都是核心线程，这个时候线程是否会退出，取决于allowCoreThreadTimeOut
拒绝策略
常用拒绝策略
CallerRunsPolicy （调用者运行策略）
功能当触发拒绝策略时，只要线程池没有关闭，就由提交任务的当前线程处理
使用场景一般在不允许失败的、对性能要求不高、并发量较小的场景下使用，因为线程池一般情况下不会关闭，也就是提交的任务一定会被运
行，但是由于是调用者线程自己执行的，当多次提交任务时，就会阻塞后续任务执行，性能和效率自然就慢了
AbortPolicy（默认：中止策略）
功能当触发拒绝策略时，直接抛出拒绝执行的异常，中止策略的意思也就是打断当前执行流程
使用场景这个就没有特殊的场景了，但是一点要正确处理抛出的异常 ThreadPoolExecutor中默认的策略就是AbortPolicy
DiscardPolicy（丢弃策略）
功能直接静悄悄的丢弃这个任务，不触发任何动作
使用场景如果你提交的任务无关紧要，你就可以使用它。因为它就是个空实现，会悄无声息的吞噬你的的任务。所以这个策略基本上不用了
DiscardOldestPolicy（弃老策略）
功能如果线程池未关闭，就弹出队列头部的元素，然后尝试执行
使用场景这个策略还是会丢弃任务，丢弃时也是毫无声息，但是特点是丢弃的是老的未执行的任务，而且是待执行优先级较高的任务
第三方实现的拒绝策略
dubbo 当dubbo的工作线程触发了线程拒绝后，主要做了三个事情：1）输出了一条警告级别的日志；2）输出当前
线程堆栈详情；3）继续抛出拒绝执行异常，使本次任务失败，这个继承了JDK默认拒绝策略的特性
Netty
Netty中的实现很像JDK中的CallerRunsPolicy，舍不得丢
弃任务。不同的是，CallerRunsPolicy是直接在调用者线程
执行的任务。而 Netty是新建了一个线程来处理
Netty的实现相较于调用者执行策略的使用面就可以扩展到支持高效率高性能的场景了。但是也
要注意一点，Netty的实现里，在创建线程时未做任何的判断约束，也就是说只要系统还有资源
就会创建新的线程来处理，直到new不出新的线程了，才会抛创建线程失败的异常
activeMq activeMq中的策略属于最大努力执行任务型，当触发拒绝策略时，在尝试一分钟的时间重新将
任务塞进任务队列，当一分钟超时还没成功时，就抛出异常
CompletionService
是什么将Executor和BlockingQueue的功能融合在一起.可以将Callable任务提交给它来执行,然后使用类似与队列操作的take和poll等方法来获得已完成的结果
目标任务先完成可优先获取到，即结果按照完成先后顺序排序
TODO
CompletableFuture TODO
如何合理设置线程池大小
几个角度分析
任务的性质
CPU密集型任务应配置尽可能小的线程，如配置CPU个数+1的线程数
IO密集型任务应配置尽可能多的线程，因为IO操作不占用CPU，不要让CPU闲下来，
应加大线程数量，如配置两倍CPU个数+1
混合型的任务如果可以拆分，拆分成IO密集型和CPU密集型分别处理，前提是两者运
行的时间是差不多的，如果处理时间相差很大，则没必要拆分了
任务的优先级高、中、低
任务的执行时间长、中、短
任务的依赖性
若任务对其他系统资源有依赖，如某个任务依赖数据库的连接返
回的结果，这时候等待的时间越长，则CPU空闲的时间越长，那
么线程数量应设置得越大，才能更好的利用CPU
当然具体合理线程池值大小，需要结合系统实际
情况，在大量的尝试下比较才能得出
估算合理值的公式
最佳线程数目 = （（线程等待时间+线程CPU时间）/线程CPU时间）* CPU数目最佳线程数目 = （线程等待时间与线程CPU时间之比 + 1）* CPU数目
线程等待时间所占比例越高，需要越多线程。线程CPU时间所占比例越高，需要越少线程。以上公式与之前的CPU和IO密集型任务设置线程数基本吻合
QA
线程池被创建后里面有线程吗？如果没有的话，
你知道有什么方法对线程池进行预热吗？
线程池被创建后如果没有任务过来，里面是不会有线程的。如果需要预热的话可以调用下面的两个方法：
prestartCoreThread()和prestartAllCoreThreads()
核心线程数会被回收吗？需要什么设置？ 核心线程数默认是不会被回收的，如果需要回收核心线程数，需要调用下面的方法：allowCoreThreadTimeOut()
6. 类加载 Class.forName和classloader的区别
相同 java中Class.forName 和ClassLoader.loadClass 都可用来对类进行加载
不同
class.forName()前者除了将类的.class文件加载到jvm中之外，还会对类进行解释，执行类中的static块
classLoader只干一件事情，就是将.class文件加载到jvm中，不会执行static中的内容,只有在newInstance才会去执行static块
7. 其它
对象在内存中分为三部分 Java对象
对象头
Mark World
指向类的指针
数组长度
实例数据
对齐填充字节
JDK和CGLib动态代理
Jdk动态代理
利用拦截器（必须实现InvocationHandler接口）加上反射机制生成一个代理接口的匿名类，在调用具体方法前调用InvokeHandler来处理
如果想要实现JDK动态代理那么代理类必须实现接口
Cglib动态代理
如果想要使用CGlib动态代理，那么代理类不能使用final修饰类和方法
利用ASM框架，对代理对象类生成的class文件加载进来，通过修改其字节码生成子类来进行代理
CGLib所创建的动态代理对象在实际运行时候的性能要比JDK动态代理高不少，有研究表明，大概要高10倍
CGLib在创建对象的时候所花费的时间却比JDK动态代理要多很多，有研究表明，大概有8倍的差距
对于singleton的代理对象或者具有实例池的代理，因为无需频繁的创建代理对象，所以比较适合采用CGLib动态代理，反正，则比较适用JDK动态代理
session和cookie的区别
cookie
①cookie是创建于服务器端
②cookie保存在浏览器端
③cookie的生命周期可以通过cookie.setMaxAge(2000);来设置，如果没有设置setMaxAge, 则cookie的生命周期当浏览器关闭的时候，就消亡了
④cookie可以被多个同类型的浏览器共享,可以把cookie想象成一张表
比较
同步异步，阻塞和非阻塞
4个概念
同步异步
同步
顺序是连续的任务执行的顺序是连续的，一个任务完成后才开始下一个任务。
指收到 I/O 请求后，系统不会立刻响应应用程序；等到处理完成，系统才会通过系
统调用的方式，告诉应用程序 I/O 结果
系统调用 read 是同步读，所以，在没有得到磁
盘数据前，read 不会响应应用程序
异步
指的是I/O操作的启动和处理是分离的，I/O请求被发起后，程序可以立即执行后续操作，不必等待I/O操作完成
指收到 I/O 请求后，系统会先告诉应用程序 I/O 请求已经收到，随后再去异步处
理；等处理完成后，系统再通过事件通知的方式，告诉应用程序结果
aio_read 是异步读，系统收到 AIO 读请求后不等处理就返回
了，而具体的 read 结果，再通过回调异步通知应用程序
区别
同步模式强调按顺序执行任务，可能会导致等待I/O操作期间的时间浪费。
异步模式允许程序在等待I/O操作的同时执行其他任务，提高效率。
阻塞非阻塞
阻塞阻塞指的是在等待I/O操作完成期间，当前线程或
进程会被挂起（阻塞），无法执行其他任务。
send() 直接向套接字发送数据时，如果套接字没有设置 O_NONBLOCK
标识，那么 send() 操作就会一直阻塞，当前线程也没法去做其他事情
非阻塞在等待I/O操作时，当前线程或进程不会被完全挂起。它会立即返回，可以执行
其他操作，但需要不断检查I/O操作是否完成（通常通过轮询）。
如果使用 epoll，系统会告诉你这个套接字的状态，那就可以用非阻塞的方式使用。
当这个套接字不可写的时候，你可以去做其他事情，比如读写其他套接字
区别阻塞与非阻塞描述的是在等待I/O操作完成时程序的状态。阻塞意味着挂起，非阻塞则意味着继续执行其他任务
区别阻塞 / 非阻塞针对的是 I/O 调用者（即应用程序），而同步 / 异步针对的是 I/O 执行者（即系统）
组合
同步阻塞
一个Web服务器使用同步阻塞I/O处理HTTP请求。服务器在读取整个请求之前不会做其他工作。
示例
InputStream stream = new FileInputStream("file.txt");
int data = stream.read(); // 阻塞，直到数据可读
future.get()
用于等待任务的完成并获取任务的结果。
future.get() 在Java中的应用，它通常与线程池（如ExecutorService）结合使用，以实现并发任务处理。
这是一个同步阻塞的例子，因为调用 future.get() 的线程会等待直到Future任务完成
尽管任务是在另一个线程中异步执行的，但 future.get() 是一个同步阻塞的调用，因为它会使主线程等待直到任务完成
同步非阻塞
在处理I/O时，程序会周期性地检查数据是否已经准备好，同时能执行其他任务。
示例
//Java NIO（New I/O）, Java NIO库 (java.nio).
SocketChannel socketChannel = SocketChannel.open();
socketChannel.configureBlocking(false); // 设置为非阻塞模式
int bytesRead = socketChannel.read(buffer); // 非阻塞，立即返回
异步阻塞
通常我们不会说异步阻塞，因为异步的本质是不阻塞当前线程。
示例这种类型比较罕见，通常异步操作都是非阻塞的。不过，理
论上可以通过在异步操作后立即等待结果来实现。
//在JavaScript中使用Promise，但紧接着使用await。
//中间件/库: JavaScript的fetch API。
async function fetchData() {
 let data = await fetch(url); // 异步，但使用await使其变为阻塞
 return data;
}
异步非阻塞
I/O操作完成后，通常通过回调函数、事件或Promise等机制来通知程序示例：在Web编程中，JavaScript常用的异步非阻塞模式，如使
用Promises或Async/Await处理HTTP请求。
示例 javaScript中处理HTTP请求
斐波那契
动态规划：斐波那契序列是动态规划（Dynamic Programming）的经典例题之一，用于教学和实践中，展示如何将一个大问题分解成小问题，通过解决并记住小问题的答案来有效
地解决大问题，从而提高算法效率。
计算机图形学：在计算机图形学中，斐波那契序列用于创建自然现象的模拟效果，如生成具有自然外观的树叶排列、花瓣分布等，通过模拟自然界中斐波那契序列的排列，创造出
真实感更强的虚拟图像。
负载均衡：在计算机网络的负载均衡技术中，斐波那契散列有时被用来优化资源的分配。通过斐波那契序列的特性，可以有效分散请求，减少服务器间的负载不均现象
加密算法：斐波那契序列也被应用于某些加密算法中，利用其数列的复杂性来增强密码的安全性。斐波那契序列的不规则性使得解码变得更加困难。
软件工程：在软件工程的估算和规划过程中，斐波那契数列有时被用于敏捷开发中的规划扑克（Planning Poker）。这是一种估算工作量的技术，斐波那契数列中的数值被用作估
算的单位，帮助团队成员以相对量化的方式讨论和一致对任务复杂度的认识。
搜索算法优化：斐波那契搜索算法是一种在有序数组中查找元素的高效算法，利用斐波那契数列来分割数组，减少比较次数，从而加快搜索速度。
在生活中，斐波那契序列的应用同样广泛且令人着迷。从自然界的排列顺序，如向日葵的花瓣排列、松果的螺旋图案，到人类艺术与建筑中的美学设计，如著名的帕尔特农神庙，
斐波那契序列无处不在，揭示了自然与文化的和谐共鸣。
在艺术创作中，斐波那契序列也被用作构图的黄金比例，创造出既平衡又吸引人的作品。摄影师和画家们通过这一比例安排画面的元素，让观赏者的眼睛自然而然地被引向作品的
焦点。
关键字
基础类型占用内存大小
1字节 boolean和byte
2字节 short和char
4字节 int和float
8字节
long和double
Reference
transient
属性不需要被序列化，这些信息对应的变量就可以加上transient关键字
transient关键字只能修饰变量，而不能修饰方法和类
一个静态变量不管是否被transient修饰，均不能被序列化
对象的序列化
Serializable接口，则所有的序列化将会自动进行
Externalizable接口，则没有任何东西可以自动序列化，需要在writeExternal方法中进行手工指定所要序列化的变量，这与
是否被transient修饰无关。因此第二个例子输出的是变量content初始化的内容，而不是null
String 不可变类，属性value为不可变数组，不存在字符数组扩容一说。字符串拼接，截取，都会生成一个新的对象。频繁操作字符串效率低下，因为每次都会生成新的对象
不可变类
是什么所谓的不可变类是指这个类的实例一旦创建完成后，就不能改变其成员变量值
好处
安全
在多线程环境(multithreading)是线程安全的,一个 String 实例可以在不同的线程中共享。这避免了使用同步(synchronization)来保护线程安全
String被用在了classloader 类加载器中，不可变性提供了正确的类由Classloader加载的安全性。例如，假设您尝试加载java.sql.Connection
类的实例，但引用的值更改为myhacked.Connection类，可以对数据库执行有害的操作
性能
Java Runtime 就可以节省大量的 Java 堆空间，因为不同的 String 变量可以在String Pool中引用相同的 String 变量(实际String对象的值)
它的 hashcode 在创建时就被缓存了，它不需要再次计算,并且它的处理速度比其他 Object对象要快
StringBuilder 类内部维护可变长度char[] ，初始化数组容量为16，存在扩容，其append拼接字符串方法内部调用System的native方法，进行数组的拷贝，不
会重新生成新的StringBuilder对象
StringBuffer 类内部维护可变长度char[]，基本上与StringBuilder一致，但其为线程安全的字符串操作类，大部分方法都采用了Synchronized关键字修改，以
此来实现在多线程下的操作字符串的安全性
Integer.parseInt原理
前置条件判断：字符串非空；2<radix<36（radix表示进制）
符号位和范围判断： Integer.MIN_VALUE<result<Integer.MAX_VALUE。MIN_VALUE = 0x80000000;MAX_VALUE = 0x7fffffff，即
[-2^31,2^31-1]一个int值占4个字节byte，一个字节是8位bit（即8个二进制位），所以int型占32位。
核心代码
while (i < len) {
digit = Character.digit(s.charAt(i++),radix);
result *= radix;
result -= digit; //digit：第i位上的字符转换为对应数字
}
return negative ? result : -result;
Unsafe
Unsafe类使Java拥有了像C语言的指针一样操作
内存空间的能力，一旦能够直接操作内存
手动GC 不受jvm管理，也就意味着无法被GC，需要我们手动GC，稍有不慎就会出现内存泄漏
偏移量要自己计算 Unsafe的不少方法中必须提供原始地址(内存地址)和被替换对象的地址，偏移量要自己计算，一旦出现问题就是JVM崩溃级别的异常，会导致整
个JVM实例崩溃，表现为应用程序直接crash掉
直接操作内存也意味着其速度更快，在高并发的条件之下能够很好地提高效率
serialVersionUID
在没有定义serialVersionUID的时候，会调用computeDefaultSUID 方
法，生成一个默认的serialVersionUID
先使用一个类User1定义一个对象，该类中不定义serialVersionUID，将其写入文件。然后我们修改User1类，向其中增加一个属性。在
尝试将其从文件中读取出来，并进行反序列化。执行结果：同样，抛出了InvalidClassException，并且指出两个serialVersionUID不同
在兼容性升级中，在修改类的时候，不要修改serialVersionUID的原因。除非是完全不兼容的两个版本。所
以， serialVersionUID其实是验证版本一致性的
如果增加兼容的字段， serialVersionUID保持不变
如果增加不兼容的字段，serialVersionUID变，要通知调用方更换jar
Queue与Deque的区别
相同点 Queue以及Deque都是继承于Collection
不同
Queue Queue是FIFO的单端队列
Deque
Deque是Queue的子接口
Deque是双端队列 LinkedList内部封装的是双向链表数据结构
聚合与组合的区别
聚合
聚合关系是关联关系的一种，聚合是整体和个体之间的关系。Aggregation is used to represent ownership or a
whole/part relationship。一般的表现形式是一个类持有另一个类作成员变量来体现，但绝对不可以New它
聚合关系当一个类destory的时候，持有的另一个类不会受到影响。聚合是"has a"
组合
组合关系是关联关系的一种，是比聚合关系强的关系。它要求普通的聚合关系中代表整体的对象负责代表部分对象的生命周期，组合关系是不能共享的
一个类destory的时候，持有的另一个类一定也会被destory。是个体的生活周期被整体所决定
聚合只是引用，组合而是去New对象了，赋值了
java8的两个冒号是什么意思
是什么 Java 8引入的"两个冒号"操作符(::)是方法引用的语法。它用于直接引用类或对象的方法，通常用在函数式接口的上下文中，比如在Lambda表达式的场景里
方法引用的几种类型
静态方法引用（static method）语法：classname::methodname 例如：Person::getAge
对象的实例方法引用语法：instancename::methodname 例如：System.out::println
对象的超类方法引用语法： super::methodname
类构造器引用语法： classname::new 例如：ArrayList::new
数组构造器引用语法： typename[]::new 例如： String[]:new
伪共享
Cache Line
cache line（缓存行）。缓存从内存加载数据时，并不是只加载我们请求的那部分，而是会多加载一些，例如我们想访问一个 int，只有 4 字节，但缓存会一
次性加载 64 字段（不同机器不同）。缓存每次处理的这一“块”数据，就叫 cache line
MESI 协议
现代 CPU 体系中，一般每个核都单独配备了自己的（L1）缓存，为了保证多个 CPU 在读写缓存时保证整体数据的一致性，通常需要使用缓存一
致性协议，MESI 就是其中一种
MESI 协议可以简单理解为“踢人协议”，如果一个 CPU 写数据到缓存里，则需要“踢”掉其它缓存里的副本
什么原因造成的
CPU缓存，其中L1是L2的子集，L2是L3的子集，L1到L3缓存容量依次增大，查找耗时依次增大，CPU查找顺序依次是L1、L2、L3、主存。L1与CPU core
对应，是单核独占的，不会出现其他核修改的问题。一般L2也是单核独占。而L3一般是多核共享，可能操作同一份数据，那么就有可能出问题
缓存行的大小通常是64字节，这意味着即使只操作1字节的数据，CPU最少也会读取这个数据所在的连续64字节数据
根据主流CPU为保证缓存有效性的MESI协议的简单理解，如果一个核正在使用的数据所在的缓存行被其他核修改，那么这个缓存行会失效，需要重新读取缓存
伪共享的发生有两个条件
两个变量在同一个缓存行里。通常是一个类/结构体的两个 field，或是同一个数组的相邻元素
不同线程同时对两个地址读写。因此通常发生在高并发的场景下
是什么如果多个核的线程在操作同一个缓存行中的不同变量数据，那么就会出现频繁的缓存失效，即使在代码层面看这两个线程操作的数据之间完全没
有关系。这种不合理的资源竞争情况学名伪共享（False Sharing），会严重影响机器的并发执行效率
解决
增加填充(Padding)
百度的snowflake实现中使用的
PaddedAtomicLong
volatile long p1, p2, p3, p4, p5, p6 = 7L;
public long sumPaddingToPreventOptimization() {
 return p1 + p2 + p3 + p4 + p5 + p6;
}
Java8及以上的版本原生支持避免伪共享，可以使用
@Contended注解
@Contended 注解会增加目标实例大小，要谨慎使用。默认情况下，除了 JDK 内部的类，JVM 会忽略该注解。要应用代码支持的话，要设
置 -XX:-RestrictContended=false，它默认为 true（意味仅限 JDK 内部的类使用）。当然，也有个 –XX: EnableContented 的配置参数
使用无锁数据结构例如原子变量或无锁队列，以避免伪共享
怎么确定性能问题是否是由伪共享造成的
性能分析工具
Linux下的perf
使用支持识别伪共享问题的专业工具，如Intel的VTune Amplifier
Java的JMH（Java Microbenchmark Harness）中的缓存命中率和延迟测试
性能对比测试不同数量的CPU核心
实验验证
例
可见多个线程操作在同一Cache Line上的不同数
据，相互竞争同一Cache Line，导致线程彼此牵
制影响，变成了串行程序，降低了并发性
数据X、Y、Z被加载到同一Cache Line中，线程A在Core1修改X，线程B在Core2上修改Y。根据
MESI缓存一致性协议，假设是Core1是第一个发起写操作的CPU核，Core1上的L1 Cache Line由S
（共享）状态变成M（修改，脏数据）状态，然后告知其他的CPU核，图例则是Core2，引用同一
地址的Cache Line已经无效了；当Core2发起写操作时，首先导致Core1将X写回主存，而后才是
Core2从主存重新读取该地址内容以便后面的修改
包冲突时，依赖的是低版本还是高版本
最短路径优先，其次最先声明优先
最短路径优先原则
A -> X -> Y -> Z(2.5)
B -> X -> Z(2.0)
生效的是Z(2.0)这个版本。因为他的路径更加短
最先声明优先原则
A -> Z(3.0)
B -> Z(2.5)
A最先声明，所以传递过来的Z选择用3.0版本的
解决冲突
排除法 idea的插件Meavn Helper上exclude
版本锁定法公司的项目一般都会有父级pom，你想指定哪个版本只需要在你项目的父POM中(当然在本工程内也可以)定义指定版本号的依赖Jar包。锁定版
本法可以打破2个依赖传递的原则，优先级为最高
如何写一个干净依赖关系的POM文件
1、尽量在父POM中定义<dependencyManagement>，来进行本项目一些依赖版本的管理，这样可以从很大程度上解决一定的冲突
2、如果是提供给别人依赖的Jar包，尽可能不要传递依赖不必要的Jar包
3、使用mvn dependency:analyze-only命令用于检测那些声明了但是没被使用的依赖，如有有一些是你自己声明的，那尽量去掉
4、使用mvn dependency:analyze-duplicate命令用来分析重复定义的依赖，清理那些重复定义的依赖
java中JDK、JRE和JVM的之间的关系
JDK （Java Development Kit）简单理解就是Java开
发工具包，是java的核心所在
包括了JRE（Java运行环境），同时在jdk文件夹bin（通常我们配置
jdk的环境变量的根目录）目录中包含了一些Java开发工具（例如：
jconsole、javac、java、javadoc、native2ascii、jar等）
JDK=JRE+Java开发工具（编译器、反编译器、调试器等）
JRE
(Java Runtime Environment)，即Java运行环
境，支持Java程序运行的标准环境，包含JVM标
准实现及Java核心类库
JRE中包含了Java virtual machine（JVM），
runtime class libraries和Java application
launcher，这些是运行Java程序的必要组件
JVM
Java虚拟机，运行在操作系统之上，存在于内
存中，与内存打交道，与硬件没有直接交互，
是Java语言实现跨平台的核心
数组拷贝的内存分配过程
怎么存放
Java 中的每一个数组存储的数据类型是一致的
Java 的数组的确是放在一块连续内存里的，否则不可能做到在 O(1) 时间复杂度内存取元素，每个元素占用相同大小的内存空间。当你知道要访问的元素的索
引时，可以直接计算出该元素在内存中的位置
基本类型的数据都非常小，可以直接放在数组里，这跟 C 里面的数组是一样的
但引用类型的对象就不一样了，存在数组里的都只是引用，不是真正的对象数据。我们通过数组拿到的还是引用，真正的对象分散地存在堆里，并不是连续的
数组是静态的 Java 数组是静态的，即当数组被初始化之后，该数组所占的内存空间、数组长度都是不可变的。Java 程序中的数组必须经过初始化才可使用。所谓初始
化，即创建实际的数组对象，也就是在内存中为数组对象分配内存空间，并为每个数组元素指定初始值
数组的初始化有以下两种方式
静态初始化初始化时由程序员显式指定每个数组元素的初始
值，由系统决定数组长度
动态初始化初始化时程序员只指定数组长度，由系统为数组
元素分配初始值
一旦初始化完成，该数组的长度就不可改变
链表和数组使用场景
数组
随机访问强需求场景当需要频繁地通过索引直接访问元素时，数组是更好的选择，因为它提供了 O(1) 的访问时间复杂度
已知固定大小的数据集如果你事先知道数据集的大小，数组是一个简单而有效的选择。尽管动态数组（如Java中的ArrayList）可以
在运行时扩容，但其底层实现仍然基于数组，且扩容操作有一定的性能开销
性能敏感的应用由于数组的数据是连续存储的，这有利于CPU缓存机制，可以提高遍历数据的速度
链表
动态数据集当数据集的大小在运行时频繁变化，且无法预先确定时，链表能够灵活地进行元素的添加和删除，且这些操作通常具有
O(1) 的时间复杂度（假设已有指向插入或删除位置的指针）
频繁插入和删除操作链表能提供更高的效率。数组在这些操作下需要移动元素以保持连续性，而链表只需修改前驱或后继节点的指针
内存使用灵活性链表通过节点的指针连接彼此，不要求物理存储空间的连续性，这使得在内存分配上更加灵活。对于大型对象和结构，使用链表可以减少内存碎片
synchronized内部轻量级和重量级锁是什么？区
别是什么？
为什么需要Session？
Session的需求源于HTTP协议的无状态性。HTTP是一种无状态的协议
Session作用
用户身份验证在用户登录后，服务器需要跟踪用户的登录状态。通过Session可以在服务器端存储用户的登录信息，确保用户在访问网站的不同页面时，不需要重复登录
存储用户状态 Session可以用来存储用户在整个网站访问期间的状态信息，如用户的偏好设置、购物车内容等
安全性相比于将状态信息存储在客户端（例如使用Cookie），Session存储在服务器端，可以有效减少敏感信息泄露的风险。服务
器可以对Session数据进行加密处理，提高安全性
提高用户体验 Session使得开发者可以为用户提供定制化的交互体验，如基于用户过往行为推荐内容，或保存用户的使用习惯设置
什么是Dockerfile
Dockerfile是一个文本文件，它包含了一系列的指令和参数，用于定义如何构建Docker镜像。每一条指令都会在镜像中创建一
个新的层，并对镜像进行修改。这些指令包括但不限于添加文件、设置环境变量、运行命令和配置启动时执行的命令
反射是什么
功能
动态创建对象通过反射机制，可以在运行时动态创建任何类的对象实例，即使在编译时不知道类的确切类型
获取和设置对象的属性反射允许程序动态访问和修改对象的字段，即使这些字段在类定义中是私有的
动态调用方法可以在运行时调用任何对象的方法，而无需在编译时确定要调用的具体方法
获取类型信息反射允许程序获取任何对象的类信息，如类的名字、方法信息、字段信息等
使用场景
开发通用框架如Spring等框架使用反射来进行依赖注入，动态地创建对象和调用对象的方法
数据映射处理如在ORM（对象关系映射）框架中，反射用于动态地将数据库中的数据映射到对象属性上
动态代理利用反射生成动态代理类，进行方法拦截和增强处理
缺点
性能开销反射操作相比直接的Java代码执行有更高的性能开销，因为它需要在运行时分析类的元数据、访问权限检查等
安全问题反射可以访问类的私有成员和方法，可能会破坏封装性，增加安全风险
future. get()
用于获取异步操作的结果。它将阻塞当前线程，直到结果可用或发生超时
存在几个关键的问题
阻塞性质调用者线程会被挂起，直到异步操作完成并且结果可用，或者直到发生异常。这
种阻塞性质意味着它可能会导致资源浪费
缺乏灵活性缺少对异步操作的细粒度控制。例如，你不能直接表达任务之间的依赖关系，
也无法组合多个Future进行流式处理或变换
异常处理如果异步操作失败，Future.get()会抛出ExecutionException。调用者需要处
理这个异常，但原始异常信息会被封装在ExecutionException中
超时处理合理设置这个超时值可能很困难。超时过短可能导致频繁的超时异常，而超时过
长则可能导致资源被不必要地占用
Java 8引入了CompletableFuture，提供了更加灵活和强大
的异步编程模型。CompletableFuture支持非阻塞获取结
果、结果组合、转换、以及利用回调处理异步操作的完成。这
使得开发者能够以更加声明式和函数式的方式来处理复杂的异
步逻辑，同时避免了Future.get()方法的一些主要弊端
如何实现Java中所有类默认继承Object类呢？
所有的类都隐式地继承自 java.lang.Object 类，这是由 Java 语言的设计和编译器自动实现的。具体来说，如果一个类没有明确指定它的父类（即
没有使用 extends 关键字明确继承某个类），那么这个类默认继承自 Object 类。这一设计是 Java 语言的一部分，确保了所有的 Java 对象都具
有 Object 类定义的基本方法，如 toString(), equals(Object obj), hashCode(), getClass(), notify(), notifyAll(), 和 wait() 等
实现机制一机制的实现是内嵌在 Java 语言规范和 Java 编
译器中的
编译器处理 JDK 6之前是编译器处理，JDK 7之后是虚拟机处理
JVM处理
Java 的finalize() 方法
做什么 finalize() 方法是 java.lang.Object 类的一个方法，每个类都继承了这个方法。它设计用来允许一个对象在被垃圾收集器回收之前执行清理操作，比如释放非Java资源（如文件句柄或数据库连接）
注在 Java 9 开始已被标记为废弃（deprecated），原因包括不确定性、性能问题和安全风险
原理
调用时机垃圾收集器决定一个对象没有更多的引用，即对象变成了垃圾时，finalize() 方法会被自动调用。然而，垃圾收集器何时运行以及
是否运行是不确定的，因此 finalize() 方法的执行时间也是不确定的
执行条件一个对象的 finalize() 方法在其生命周期中最多被调用一次
注意点如果在 finalize() 方法执行期间对象重新获得了对某个引用（例如，被赋值给某个类变量），那么在下一次垃圾收集时，finalize() 方法不会被再次调用
正确使用方法 finalize() 方法的缺点，推荐的做法是避免使用
它。Java 提供了更好的替代方案
使用 try-with-resources 管理资源
try (ResourceType resource = new ResourceType()) {
 // 使用资源
} catch (Exception e) {
 // 处理异常
Java 9 引入了 java.lang.ref.Cleaner 类，使用 Cleaner 或 PhantomReference 管理资源释放